

Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun

Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

*Rectified activation units (rectifiers) are essential for state-of-the-art neural networks. In this work, we study rectifier neural networks for image classification from two aspects. First, we propose a Parametric Rectified Linear Unit (PReLU) that generalizes the traditional rectified unit. PReLU improves model fitting with nearly zero extra computational cost and little overfitting risk. Second, we derive a robust initialization method that particularly considers the rectifier nonlinearities. This method enables us to train extremely deep rectified models directly from scratch and to investigate deeper or wider network architectures. Based on our PReLU networks (PReLU-nets), we achieve **4.94%** top-5 test error on the ImageNet 2012 classification dataset. This is a 26% relative improvement over the ILSVRC 2014 winner (GoogLeNet, 6.66% [29]). To our knowledge, our result is the first to surpass human-level performance (5.1%, [22]) on this visual recognition challenge.*

1. Introduction

Convolutional neural networks (CNNs) [17, 16] have demonstrated recognition accuracy better than or comparable to humans in several visual recognition tasks, including recognizing traffic signs [3], faces [30, 28], and handwritten digits [3, 31]. In this work, we present a result that surpasses human-level performance on a more generic and challenging recognition task - the classification task in the 1000-class ImageNet dataset [22].

In the last few years, we have witnessed tremendous improvements in recognition performance, mainly due to advances in two technical directions: building more powerful models, and designing effective strategies against overfitting. On one hand, neural networks are becoming more capable of fitting training data, because of increased complexity (*e.g.*, increased depth [25, 29], enlarged width [33, 24],

and the use of smaller strides [33, 24, 2, 25]), new non-linear activations [21, 20, 34, 19, 27, 9], and sophisticated layer designs [29, 11]. On the other hand, better generalization is achieved by effective regularization techniques [12, 26, 9, 31], aggressive data augmentation [16, 13, 25, 29], and large-scale data [4, 22].

Among these advances, the rectifier neuron [21, 8, 20, 34], *e.g.*, Rectified Linear Unit (ReLU), is one of several keys to the recent success of deep networks [16]. It expedites convergence of the training procedure [16] and leads to better solutions [21, 8, 20, 34] than conventional sigmoid-like units. Despite the prevalence of rectifier networks, recent improvements of models [33, 24, 11, 25, 29] and theoretical guidelines for training them [7, 23] have rarely focused on the properties of the rectifiers.

In this paper, we investigate neural networks from two aspects particularly driven by the rectifiers. First, we propose a new generalization of ReLU, which we call *Parametric Rectified Linear Unit* (PReLU). This activation function adaptively learns the parameters of the rectifiers, and improves accuracy at negligible extra computational cost. Second, we study the difficulty of training rectified models that are very deep. By explicitly modeling the non-linearity of rectifiers (ReLU/PReLU), we derive a theoretically sound initialization method, which helps with convergence of very deep models (*e.g.*, with 30 weight layers) trained directly from scratch. This gives us more flexibility to explore more powerful network architectures.

On the 1000-class ImageNet 2012 dataset, our PReLU network (PReLU-net) leads to a single-model result of 5.71% top-5 error, which surpasses all existing multi-model results. Further, our multi-model result achieves **4.94%** top-5 error on the test set, which is a 26% relative improvement over the ILSVRC 2014 winner (GoogLeNet, 6.66% [29]). To the best of our knowledge, our result surpasses for the first time the reported human-level performance (5.1% in [22]) on this visual recognition challenge.

深入探究整流非线性单元：在ImageNet图像分类任务中超越人类水平性能

Kaiming He、Xiangyu Zhang、Shaoqing Ren、Jian Sun

Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

摘要

整流激活单元是当前最先进的神经网络所不可或缺的重要组成部分。在本研究中，我们从两个角度探讨了用于图像分类的整流型神经网络。首先，我们提出了一种参数化修正线性单元——*PReLU*，它对传统的整流单元进行了改进。*PReLU*几乎不会带来额外的计算开销，也不会产生过拟合的风险，同时还能提升模型的拟合效果。其次，我们设计了一种兼顾整流非线性单元特性的稳健初始化方法。借助该方法，我们可以直接从零开始训练深度极高的整流型模型，并探索更深层或更宽泛的网络架构。基于我们所构建的*PReLU*网络，我们在*ImageNet 2012*图像分类数据集上实现了**4.94%**的前5名预测错误率，这一数值相比2014年*ILSVRC*竞赛的获胜者（*GoogLeNet*模型，错误率为6.66% [29]）有了26%的显著提升。据我们所知，这是首个在该视觉识别挑战赛中超越人类水平性能（5.1%，[22]）的成果。

1. 引言

卷积神经网络（CNN）[17, 16] 在多项视觉识别任务中展现出了优于或可与人类相媲美的识别准确率，这些任务包括识别交通标志 [3]、人脸 [30, 28]，以及手写数字 [3, 31]。在本研究中，我们得出的结果在更为通用且更具挑战性的识别任务——即1000类ImageNet数据集的分类任务上，实现了超越人类水平性能的表现 [22]。

近几年来，由于在两个技术方向上的突破——即构建更强大的模型，以及设计有效的过拟合防治策略——图像识别性能取得了显著提升。一方面，由于复杂度提高（例如）、层深度增加以及模型宽度扩大，神经网络越来越能够很好地拟合训练数据；另一方面，通过采用更小的步长、新的非线性激活函数以及更为复杂的层结构设计 [25, 29]，也能进一步提升模型的表现。

此外，通过有效的正则化技术 [12, 26, 9, 31]、大规模的数据增强[16, 13, 25, 29]，以及海量数据 [4, 22]，也能帮助模型实现更好的泛化能力。

在这些进展之中，整流器神经元 [21, 8, 20,34]，例如，也就是修正线性单元（ReLU），正是深度网络近期取得成功的关键因素之一 [16]。它不仅能够加快训练过程的收敛速度 [16]，还能带来比传统S形函数单元更好的解决方案 [21, 8, 20, 34]。尽管整流器网络已被广泛应用，但迄今为止针对模型 [33, 24, 11, 25, 29] 的改进以及相关的训练理论指导 [7, 23]，很少会着重探讨整流器本身的特性。

在本文中，我们从两个角度对神经网络展开了研究，其中核心动力便是各种修正线性单元。首先，我们提出了一种ReLU的新泛化形式，将其命名为参数化修正线性单元（PReLU）。这种激活函数能够自适应地学习修正线性单元的参数，且在几乎不增加额外计算成本的情况下就能提升模型的准确率。其次，我们探讨了训练深度极大的修正线性单元模型的难度。通过明确建模ReLU/PReLU这类修正线性单元的非线性特性，我们推导出一种具有理论依据的初始化方法，有助于直接从零开始训练的深度极深的模型（例如拥有30个权重层的模型）更快收敛。这一方法也为我们探索更强大的网络架构提供了更大灵活性。

在包含1000个类别的ImageNet 2012数据集上，我们基于PReLU构建的模型（PReLU-net）取得了5.71%的Top-5错误率，这一单模型性能已超越所有现有的多模型方案。此外，该多模型方案在测试集上的Top-5错误率为**4.94%**，相比2014年ILSVRC竞赛的获胜者GoogLeNet模型（6.66% [29]）实现了26%的相对提升。据我们所知，这一成果首次超越了此前在该视觉识别挑战赛中报道的人类水平性能（[22]中的5.1%）。

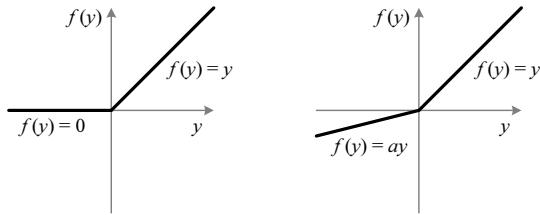


Figure 1. ReLU vs. PReLU. For PReLU, the coefficient of the negative part is not constant and is adaptively learned.

2. Approach

In this section, we first present the PReLU activation function (Sec. 2.1). Then we derive our initialization method for deep rectifier networks (Sec. 2.2). Lastly we discuss our architecture designs (Sec. 2.3).

2.1. Parametric Rectifiers

We show that replacing the parameter-free ReLU activation by a learned parametric activation unit improves classification accuracy¹.

Definition

Formally, we consider an activation function defined as:

$$f(y_i) = \begin{cases} y_i, & \text{if } y_i > 0 \\ a_i y_i, & \text{if } y_i \leq 0 \end{cases}. \quad (1)$$

Here y_i is the input of the nonlinear activation f on the i th channel, and a_i is a coefficient controlling the slope of the negative part. The subscript i in a_i indicates that we allow the nonlinear activation to vary on different channels. When $a_i = 0$, it becomes ReLU; when a_i is a learnable parameter, we refer to Eqn.(1) as *Parametric ReLU* (PReLU). Figure 1 shows the shapes of ReLU and PReLU. Eqn.(1) is equivalent to $f(y_i) = \max(0, y_i) + a_i \min(0, y_i)$.

If a_i is a small and fixed value, PReLU becomes the Leaky ReLU (LReLU) in [20] ($a_i = 0.01$). The motivation of LReLU is to avoid zero gradients. Experiments in [20] show that LReLU has negligible impact on accuracy compared with ReLU. On the contrary, our method adaptively learns the PReLU parameters jointly with the whole model. We hope for end-to-end training that will lead to more specialized activations.

PReLU introduces a very small number of extra parameters. The number of extra parameters is equal to the total number of channels, which is negligible when considering the total number of weights. So we expect no extra risk of overfitting. We also consider a channel-shared variant:

¹Concurrent with our work, Agostinelli *et al.* [1] also investigated learning activation functions and showed improvement on other tasks.

$f(y_i) = \max(0, y_i) + a \min(0, y_i)$ where the coefficient is shared by all channels of one layer. This variant only introduces a single extra parameter into each layer.

Optimization

PReLU can be trained using backpropagation [17] and optimized simultaneously with other layers. The update formulations of $\{a_i\}$ are simply derived from the chain rule. The gradient of a_i for one layer is:

$$\frac{\partial \mathcal{E}}{\partial a_i} = \sum_{y_i} \frac{\partial \mathcal{E}}{\partial f(y_i)} \frac{\partial f(y_i)}{\partial a_i}, \quad (2)$$

where $\mathcal{E}$ represents the objective function. The term $\frac{\partial \mathcal{E}}{\partial f(y_i)}$ is the gradient propagated from the deeper layer. The gradient of the activation is given by:

$$\frac{\partial f(y_i)}{\partial a_i} = \begin{cases} 0, & \text{if } y_i > 0 \\ y_i, & \text{if } y_i \leq 0 \end{cases}. \quad (3)$$

The summation $\sum_{y_i}$ runs over all positions of the feature map. For the channel-shared variant, the gradient of a is $\frac{\partial \mathcal{E}}{\partial a} = \sum_i \sum_{y_i} \frac{\partial \mathcal{E}}{\partial f(y_i)} \frac{\partial f(y_i)}{\partial a}$, where $\sum_i$ sums over all channels of the layer. The time complexity due to PReLU is negligible for both forward and backward propagation.

We adopt the momentum method when updating a_i :

$$\Delta a_i := \mu \Delta a_i + \epsilon \frac{\partial \mathcal{E}}{\partial a_i}. \quad (4)$$

Here μ is the momentum and ϵ is the learning rate. It is worth noticing that we do not use weight decay (l_2 regularization) when updating a_i . A weight decay tends to push a_i to zero, and thus biases PReLU toward ReLU. Even without regularization, the learned coefficients rarely have a magnitude larger than 1 in our experiments. Further, we do not constrain the range of a_i so that the activation function may be non-monotonic. We use $a_i = 0.25$ as the initialization throughout this paper.

Comparison Experiments

We conducted comparisons on a deep but efficient model with 14 weight layers. The model was studied in [10] (model E of [10]) and its architecture is described in Table 1. We choose this model because it is sufficient for representing a category of very deep models, as well as to make the experiments feasible.

As a baseline, we train this model with ReLU applied in the convolutional (conv) layers and the first two fully-connected (fc) layers. The training implementation follows [10]. The top-1 and top-5 errors are 33.82% and 13.34% on ImageNet 2012, using 10-view testing (Table 2).

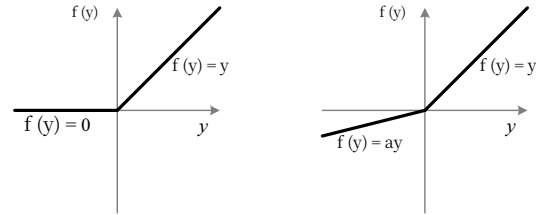


图1: ReLU与PReLU的对比。对于PReLU而言, 其负向部分的系数并非固定值, 而是通过自适应学习方式确定的。

2. 方法

在本节中, 我们首先介绍PReLU激活函数(第2.1节), 随后阐述深度整流器网络的初始化方法(第2.2节), 最后探讨相关架构设计(第2.3节)。

2.1 参数化整流器

我们证明了用经过学习的参数化激活单元替代无需参数的ReLU激活函数, 能够提升分类的准确率¹。

定义

从形式上来看, 我们将激活函数定义为:

$$f(y_i) = \begin{cases} y_i, & \text{if } y_i > 0 \\ a_i y_i, & \text{if } y_i \leq 0 \end{cases}. \quad (1)$$

此处 y_i 为第 i 通道上非线性激活函数 f 的输入, 而 a_i 则是用于控制负值部分斜率的系数。 a_i 中的下标 i 表明我们允许非线性激活函数在不同通道上有所差异。当 $a_i = 0$ 时, 它就变成了ReLU激活函数; 而当 a_i 为可学习参数时, 我们将方程(1)称为参数化ReLU (即PReLU)。图1展示了ReLU和PReLU的函数形态。方程(1)等价于 $f(y_i) = \max(0, y_i) + a_i \min(0, y_i)$ 。

如果 a_i 是一个较小且固定的数值, 那么PReLU在 [20] ($a_i = 0.01$) 情况下就会转变为泄漏ReLU函数 (LReLU)。采用LReLU的目的是为了避免梯度为零的情况。[20] 中的实验结果表明, 与ReLU相比, LReLU对模型准确率的影响几乎可以忽略不计。相反, 我们的该方法会与整个模型一起自适应地学习PReLU的各个参数, 期望通过端到端训练来获得更为专业的激活函数形式。

PReLU仅引入了极少量的额外参数。这些额外参数的数量恰好等于通道总数, 在考虑整体权重数量时可以忽略不计。因此我们认为它不会带来过拟合的风险。此外, 我们还将一种通道共享式的变体纳入了研究范围。

¹在与我们开展研究的同时, Agostinelli 等人也对学习激活函数进行了探索, 并在其他任务上取得了优化效果。

$f(y_i) = \max(0, y_i) + a \min(0, y_i)$, 其中该系数会被同一层的所有通道共享。这种变体仅在每一层中增加一个额外的参数。

优化

PReLU可以通过反向传播进行训练, 同时与其他层一同优化。 $\{a_i\}$ 中的更新公式其实可直接从链式法则推导得出。某一层的 a_i 的梯度表达式为:

$$\frac{\partial \mathcal{E}}{\partial a_i} = \sum_{y_i} \frac{\partial \mathcal{E}}{\partial f(y_i)} \frac{\partial f(y_i)}{\partial a_i}, \quad (2)$$

其中 $\mathcal{E}$ 代表目标函数, 而 $\frac{\partial \mathcal{E}}{\partial f(y_i)}$ 这一项则是从更深层传递而来的梯度。激活函数的梯度则由下式给出:

$$\frac{\partial f(y_i)}{\partial a_i} = \begin{cases} 0, & \text{if } y_i > 0 \\ y_i, & \text{if } y_i \leq 0 \end{cases}. \quad (3)$$

该求和 $\sum_{y_i}$ 是对特征图的所有位置进行的。在采用通道共享式的结构时, a 的梯度表现为 $\frac{\partial \mathcal{E}}{\partial a} = \sum_i \sum_{y_i} \frac{\partial \mathcal{E}}{\partial f(y_i)} \frac{\partial f(y_i)}{\partial a}$, 此处 $\sum_i$ 是对该层所有通道求和得到的。无论是前向传播还是反向传播, PReLU所带来的时间复杂度都可以忽略不计。

在更新 a_i 时, 我们采用了动量法。

$$\Delta a_i := \mu \Delta a_i + \epsilon \frac{\partial \mathcal{E}}{\partial a_i}. \quad (4)$$

此处 μ 代表动量值, ϵ 代表学习率。值得注意的是, 在更新 a_i 时我们并未使用权重衰减(即 l_2 正则化) 机制。因为权重衰减往往会将 a_i 的值推向零, 进而使PReLU激活函数趋向于ReLU激活函数。即便不采用正则化, 在我们的实验中学习得到的系数的幅度也很少超过1。此外, 我们也没有对 a_i 的取值范围进行限制, 这样激活函数就可以呈现非单调特性。在整篇论文中, 我们都统一使用 $a_i = 0.25$ 作为模型的初始值。

比较实验

我们选择了一个具有14个权重层的深度较高但效率良好的模型来进行对比实验。该模型曾在 [10]中被研究过(即 [10]中的模型E), 其架构详见表1。之所以选用这个模型, 是因为它足以代表一类非常深的模型, 同时也能确保实验的可行性。

作为基准模型, 我们在该模型的卷积层以及前两层全连接层中采用ReLU激活函数进行训练, 具体的训练实现方式遵循[10]。在采用10视图测试的ImageNet 2012数据集(表2)上, 该基准模型的Top-1错误率和前五名错误率分别为33.82%和13.34%。

		learned coefficients	
layer		channel-shared	channel-wise
conv1	7×7, 64, /2	0.681	0.596
pool1	3×3, /3		
conv2 ₁	2×2, 128	0.103	0.321
conv2 ₂	2×2, 128	0.099	0.204
conv2 ₃	2×2, 128	0.228	0.294
conv2 ₄	2×2, 128	0.561	0.464
pool2	2×2, /2		
conv3 ₁	2×2, 256	0.126	0.196
conv3 ₂	2×2, 256	0.089	0.152
conv3 ₃	2×2, 256	0.124	0.145
conv3 ₄	2×2, 256	0.062	0.124
conv3 ₅	2×2, 256	0.008	0.134
conv3 ₆	2×2, 256	0.210	0.198
spp	{6, 3, 2, 1}		
fc ₁	4096	0.063	0.074
fc ₂	4096	0.031	0.075
fc ₃	1000		

Table 1. A small but deep 14-layer model [10]. The filter size and filter number of each layer is listed. The number /*s* indicates the stride *s* that is used. The learned coefficients of PReLU are also shown. For the channel-wise case, the average of $\{a_i\}$ over the channels is shown for each layer.

	top-1	top-5
ReLU	33.82	13.34
PReLU, channel-shared	32.71	12.87
PReLU, channel-wise	32.64	12.75

Table 2. Comparisons between ReLU and PReLU on the small model. The error rates are for ImageNet 2012 using 10-view testing. The images are resized so that the shorter side is 256, during both training and testing. Each view is 224×224. All models are trained using 75 epochs.

Then we train the same architecture from scratch, with all ReLUs replaced by PReLUs (Table 2). The top-1 error is reduced to 32.64%. This is a **1.2%** gain over the ReLU baseline. Table 2 also shows that channel-wise/channel-shared PReLUs perform comparably. For the channel-shared version, PReLU only introduces 13 extra free parameters compared with the ReLU counterpart. But this small number of free parameters play critical roles as evidenced by the 1.1% gain over the baseline. This implies the importance of adaptively learning the shapes of activation functions.

Table 1 also shows the learned coefficients of PReLUs for each layer. There are two interesting phenomena in Table 1. First, the first conv layer (conv1) has coefficients (0.681 and 0.596) significantly greater than 0. As the filters of conv1 are mostly Gabor-like filters such as edge or texture detectors, the learned results show that both positive and negative responses of the filters are respected. We be-

lieve that this is a more economical way of exploiting low-level information, given the limited number of filters (*e.g.*, 64). Second, for the channel-wise version, the deeper conv layers in general have smaller coefficients. This implies that the activations gradually become “more nonlinear” at increasing depths. In other words, the learned model tends to keep more information in earlier stages and becomes more discriminative in deeper stages.

2.2. Initialization of Filter Weights for Rectifiers

Rectifier networks are easier to train [8, 16, 34] compared with traditional sigmoid-like activation networks. But a bad initialization can still hamper the learning of a highly non-linear system. In this subsection, we propose a robust initialization method that removes an obstacle of training extremely deep rectifier networks.

Recent deep CNNs are mostly initialized by random weights drawn from Gaussian distributions [16]. With fixed standard deviations (*e.g.*, 0.01 in [16]), very deep models (*e.g.*, >8 conv layers) have difficulties to converge, as reported by the VGG team [25] and also observed in our experiments. To address this issue, in [25] they pre-train a model with 8 conv layers to initialize deeper models. But this strategy requires more training time, and may also lead to a poorer local optimum. In [29, 18], auxiliary classifiers are added to intermediate layers to help with convergence.

Glorot and Bengio [7] proposed to adopt a properly scaled uniform distribution for initialization. This is called “*Xavier*” initialization in [14]. Its derivation is based on the assumption that the activations are linear. This assumption is invalid for ReLU and PReLU.

In the following, we derive a theoretically more sound initialization by taking ReLU/PReLU into account. In our experiments, our initialization method allows for extremely deep models (*e.g.*, 30 conv/fc layers) to converge, while the “*Xavier*” method [7] cannot.

Forward Propagation Case

Our derivation mainly follows [7]. The central idea is to investigate the variance of the responses in each layer.

For a conv layer, a response is:

$$\mathbf{y}_l = \mathbf{W}_l \mathbf{x}_l + \mathbf{b}_l. \quad (5)$$

Here, $\mathbf{x}$ is a k^2c -by-1 vector that represents co-located $k \times k$ pixels in c input channels. k is the spatial filter size of the layer. With $n = k^2c$ denoting the number of connections of a response, $\mathbf{W}$ is a d -by- n matrix, where d is the number of filters and each row of $\mathbf{W}$ represents the weights of a filter. $\mathbf{b}$ is a vector of biases, and $\mathbf{y}$ is the response at a pixel of the output map. We use l to index a layer. We have $\mathbf{x}_l = f(\mathbf{y}_{l-1})$ where f is the activation. We also have $c_l = d_{l-1}$.

		学习得到的系数	
层		通道共享式	按通道
conv1卷积层	7×7, 64, /2	0.681	0.596
池化层1	3×3, /3		
conv2 ₁	2×2, 128	0.103	0.321
conv2 ₂	2×2, 128	0.099	0.204
conv2 ₃	2×2, 128	0.228	0.294
conv2 ₄	2×2, 128	0.561	0.464
pool2	2×2, /2		
conv3 ₁	2×2, 256	0.126	0.196
conv3 ₂	2×2, 256	0.089	0.152
conv3 ₃	2×2, 256	0.124	0.145
conv3 ₄	2×2, 256	0.062	0.124
conv3 ₅	2×2, 256	0.008	0.134
conv3 ₆	2×2, 256	0.210	0.198
spp	{6, 3, 2, 1}		
fc ₁	4096	0.063	0.074
fc ₂	4096	0.031	0.075
fc ₃	1000		

表1. 该表展示了一个规模虽小但深度达14层的模型 [10]的参数信息，列出了每一层的滤波器尺寸与滤波器数量。数值/*s* 表示所采用的步长 *s*，同时还给出了PReLU学习得到的系数。在按通道处理的情况下，每层的数据均为各通道上 $\{a_i\}$ 值的平均值。

	Top-1错误率	Top-5错误率
ReLU	33.82	13.34
PReLU, 通道共享式	32.71	12.87
PReLU, 按通道	32.64	12.75

表2. 小规模模型上ReLU与PReLU的性能对比。所显示的错误率是基于使用10种视图进行测试的ImageNet 2012数据集得出的；在训练和测试阶段，所有图像的短边长度都会被调整为256像素。每种视图对应 224×224个样本，所有模型均经过75个训练轮数的训练。

接着，我们重新从零开始训练相同的架构，但将所有的ReLU激活函数替换为PReLU激活函数（见表2）。如此一来，Top-1错误率降到了32.64%，相比使用ReLU的基准模型提升了1.2%。表2还显示，按通道或跨通道共享的PReLU激活函数表现相当。对于跨通道共享版本而言，与ReLU版本相比，它仅多了13个自由参数；不过正如其带来的1.1%性能提升所证明的那样，这些为数不多的自由参数却发挥着至关重要的作用。这充分体现了自适应学习激活函数形态的重要性。

表1还展示了各层PReLU的学学习得到的系数。表1中存在两个值得关注的现象。首先，第一个卷积层（conv1）的系数（0.681和0.596）显著大于0。由于conv1中的滤波器多为高斯型滤波器，例如用于检测边缘或纹理的滤波器，学学习得到的结果表明这些滤波器的正负响应都得到了保留。我们认为，鉴于滤波器的数量有限（<style id=’1’>例如</style>，仅有64个），这是一种更高效地利用低级信息的做法。其次，对于按通道处理的版本，较深的卷积层的系数通常较小。这说明随着深度的增加，激活函数的“非线性程度”逐渐提升。换言之，所构建的模型倾向于在早期阶段保留更多信息，而在较深层次则具备更强的区分能力。

我们认为，这在滤波器数量有限的条件下，是一种更高效的利用低级信息的方式（例如，仅有64个）。其次，对于按通道处理的版本，较深的卷积层的系数通常较小。这表明随着深度的增加，激活函数的“非线性程度”逐渐增强。换句话说，该模型倾向于在早期阶段保留更多信息，而在更深的层次则表现出更强的区分能力。

2.2. 整流器网络的滤波器权重初始化

与传统S形激活函数网络相比，整流器网络更容易进行训练 [8, 16, 34]。然而，不恰当的初始化仍可能阻碍高度非线性系统的学习。在本小节中，我们提出了一种稳健的初始化方法，能够消除训练极其深度的整流器网络时所面临的障碍。

目前的深度卷积神经网络大多采用从高斯分布中随机抽取的权重作为初始值 [16]。由于标准差是固定不变的（例如，在 [16]中为0.01），那些具有非常深结构（例如，包含 >8 个卷积层）的模型往往难以收敛，这一现象既得到了VGG团队的 [25] 报道，也在我们的实验中得到证实。为解决这一问题， [25] 中的研究先对包含8个卷积层的模型进行预训练，以此作为更深层次模型的初始参数。不过这种策略需要更长的训练时间，且还有可能导致局部最优解的质量下降。在 [29, 18], 中，则通过在中间层添加辅助分类器来助力模型更快收敛。

Glorot与Bengio [7]提出采用经过适当缩放的均匀分布作为初始化方式。在 [14]中，这种初始化方法被称作 “*Xavier*” 初始化。它的推导基于激活函数为线性函数的假设，而这一假设对ReLU和PReLU并不成立。

在下文中，我们会综合考虑ReLU和PReLU的特点，推导出一种在理论上更为严谨的初始化方法。实验结果表明，我们所采用的初始化方法能够让深度极深的模型（例如，包含30个卷积层或全连接层）成功收敛，而 “*Xavier*” 初始化方法则 [7] 无法实现这一点。

前向传播案例

我们的推导主要遵循 [7]中的思路，核心在于研究每层输出值的方差。

对于一个卷积层而言，其输出值即为：

$$\mathbf{y}_l = \mathbf{W}_l \mathbf{x}_l + \mathbf{b}_l. \quad (5)$$

此处， $\mathbf{x}$ 是一个 k^2c 维且长度为1的向量，用于表示 c 个输入通道中位置相邻的 $k \times k$ 像素。 k 表示该层所使用的空间滤波器尺寸。其中 $n = k^2c$ 代表某个响应的连接数量， $\mathbf{W}$ 则是一个 d 行 n 列的矩阵， d 为滤波器的总数， $\mathbf{W}$ 的每一行对应一个滤波器的权重。 $\mathbf{b}$ 是偏置量向量，而 $\mathbf{y}$ 则表示输出图中某像素处的响应值。我们使用 l 来对层进行索引。在此模型中存在 $l_l = f(\mathbf{y}_{l-1})$ 的形式，其中 f 代表激活函数值，同时还有 $c_l = d_{l-1}$ 这一项。

We let the initialized elements in W_l be mutually independent and share the same distribution. As in [7], we assume that the elements in $\mathbf{x}_l$ are also mutually independent and share the same distribution, and $\mathbf{x}_l$ and W_l are independent of each other. Then we have:

$$\text{Var}[y_l] = n_l \text{Var}[w_l x_l], \quad (6)$$

where now y_l , x_l , and w_l represent the random variables of each element in $\mathbf{y}_l$, W_l , and $\mathbf{x}_l$ respectively. We let w_l have zero mean. Then the variance of the product of independent variables gives us:

$$\text{Var}[y_l] = n_l \text{Var}[w_l] E[x_l^2]. \quad (7)$$

Here $E[x_l^2]$ is the expectation of the square of x_l . It is worth noticing that $E[x_l^2] \neq \text{Var}[x_l]$ unless x_l has zero mean. For the ReLU activation, $x_l = \max(0, y_{l-1})$ and thus it does not have zero mean. This will lead to a conclusion different from [7].

If we let w_{l-1} have a symmetric distribution around zero and $b_{l-1} = 0$, then y_{l-1} has zero mean and has a symmetric distribution around zero. This leads to $E[x_l^2] = \frac{1}{2} \text{Var}[y_{l-1}]$ when f is ReLU. Putting this into Eqn.(7), we obtain:

$$\text{Var}[y_l] = \frac{1}{2} n_l \text{Var}[w_l] \text{Var}[y_{l-1}]. \quad (8)$$

With L layers put together, we have:

$$\text{Var}[y_L] = \text{Var}[y_1] \left(\prod_{l=2}^L \frac{1}{2} n_l \text{Var}[w_l] \right). \quad (9)$$

This product is the key to the initialization design. A proper initialization method should avoid reducing or magnifying the magnitudes of input signals exponentially. So we expect the above product to take a proper scalar (e.g., 1). A sufficient condition is:

$$\frac{1}{2} n_l \text{Var}[w_l] = 1, \quad \forall l. \quad (10)$$

This leads to a zero-mean Gaussian distribution whose standard deviation (std) is $\sqrt{2/n_l}$. This is our way of initialization. We also initialize $\mathbf{b} = 0$.

For the first layer ($l = 1$), we should have $n_1 \text{Var}[w_1] = 1$ because there is no ReLU applied on the input signal. But the factor 1/2 does not matter if it just exists on one layer. So we also adopt Eqn.(10) in the first layer for simplicity.

Backward Propagation Case

For back-propagation, the gradient of a conv layer is computed by:

$$\Delta \mathbf{x}_l = \hat{W}_l \Delta \mathbf{y}_l. \quad (11)$$

Here we use $\Delta \mathbf{x}$ and $\Delta \mathbf{y}$ to denote gradients ($\frac{\partial \mathcal{E}}{\partial \mathbf{x}}$ and $\frac{\partial \mathcal{E}}{\partial \mathbf{y}}$) for simplicity. $\Delta \mathbf{y}$ represents k -by- k pixels in d channels,

and is reshaped into a $k^2 d$ -by-1 vector. We denote $\hat{n} = k^2 d$. Note that $\hat{n} \neq n = k^2 c$. $\hat{W}$ is a c -by- $\hat{n}$ matrix where the filters are rearranged in the way of back-propagation. Note that W and $\hat{W}$ can be reshaped from each other. $\Delta \mathbf{x}$ is a c -by-1 vector representing the gradient at a pixel of this layer. As above, we assume that w_l and Δy_l are independent of each other, then Δx_l has zero mean for all l , when w_l is initialized by a symmetric distribution around zero.

In back-propagation we also have $\Delta y_l = f'(y_l) \Delta x_{l+1}$ where f' is the derivative of f . For the ReLU case, $f'(y_l)$ is zero or one, and their probabilities are equal. We assume that $f'(y_l)$ and Δx_{l+1} are independent of each other. Thus we have $E[\Delta y_l] = E[\Delta x_{l+1}]/2 = 0$, and also $E[(\Delta y_l)^2] = \text{Var}[\Delta y_l] = \frac{1}{2} \text{Var}[\Delta x_{l+1}]$. Then we compute the variance of the gradient in Eqn.(11):

$$\begin{aligned} \text{Var}[\Delta x_l] &= \hat{n}_l \text{Var}[w_l] \text{Var}[\Delta y_l] \\ &= \frac{1}{2} \hat{n}_l \text{Var}[w_l] \text{Var}[\Delta x_{l+1}]. \end{aligned} \quad (12)$$

The scalar 1/2 in both Eqn.(12) and Eqn.(8) is the result of ReLU, though the derivations are different. With L layers put together, we have:

$$\text{Var}[\Delta x_2] = \text{Var}[\Delta x_{L+1}] \left(\prod_{l=2}^L \frac{1}{2} \hat{n}_l \text{Var}[w_l] \right). \quad (13)$$

We consider a sufficient condition that the gradient is not exponentially large/small:

$$\frac{1}{2} \hat{n}_l \text{Var}[w_l] = 1, \quad \forall l. \quad (14)$$

The only difference between this equation and Eqn.(10) is that $\hat{n}_l = k_l^2 d_l$ while $n_l = k_l^2 c_l = k_l^2 d_{l-1}$. Eqn.(14) results in a zero-mean Gaussian distribution whose std is $\sqrt{2/\hat{n}_l}$.

For the first layer ($l = 1$), we need not compute Δx_1 because it represents the image domain. But we can still adopt Eqn.(14) in the first layer, for the same reason as in the forward propagation case - the factor of a single layer does not make the overall product exponentially large/small.

We note that it is sufficient to use either Eqn.(14) or Eqn.(10) alone. For example, if we use Eqn.(14), then in Eqn.(13) the product $\prod_{l=2}^L \frac{1}{2} \hat{n}_l \text{Var}[w_l] = 1$, and in Eqn.(9) the product $\prod_{l=2}^L \frac{1}{2} n_l \text{Var}[w_l] = \prod_{l=2}^L n_l / \hat{n}_l = c_2 / d_L$, which is not a diminishing number in common network designs. This means that if the initialization properly scales the backward signal, then this is also the case for the forward signal; and vice versa. For all models in this paper, both forms can make them converge.

Discussions

If the forward/backward signal is inappropriately scaled by a factor β in each layer, then the final propagated signal

我们设定 W_l 中初始化后的各个元素彼此独立且具有相同的分布。与[7],中的假设一致, $\mathbf{x}_l$ 中的元素同样彼此独立且分布相同,同时 $\mathbf{x}_l$ 与 W_l 也相互独立。基于这些前提,我们可以得到如下关系:

$$\text{Var}[y_l] = n_l \text{Var}[w_l x_l], \quad (6)$$

其中 y_l 、 x_l 和 w_l 分别代表 $\mathbf{y}_l$ 、 W_l 以及 $\mathbf{x}_l$ 中各元素的yl随机变量。我们设定 w_l 的均值为零,此时多个独立变量乘积的方差即为:

$$\text{Var}[y_l] = n_l \text{Var}[w_l] E[x_l^2]. \quad (7)$$

此处 $E[x_l^2]$ 表示 x_l 平方的期望值。需要注意的是,除非 x_l 的均值为零,否则 $E[x_l^2] \neq \text{Var}[x_l]$ 的均值并不为零。对于ReLU激活函数而言,其 $x_l = \max(0, y_{l-1})$,因此也不具备零均值特性,这会导致与[7]不同的结论。

如果我们让 w_{l-1} 围绕零点呈对称分布,而 $b_{l-1} = 0$ 也满足相应条件,那么 y_{l-1} 的均值将为零且同样围绕零点对称分布。当 f 采用ReLU激活函数时,这就使得 $E[x_l^2] = \frac{1}{2} \text{Var}[y_{l-1}]$ 具有特定的数值特征。将这一结果代入方程式(7)后,即可得到相应的表达式。

$$\text{Var}[y_l] = \frac{1}{2} n_l \text{Var}[w_l] \text{Var}[y_{l-1}]. \quad (8)$$

当将 L 层组合在一起时,便可以得到如下结果:

$$\text{Var}[y_L] = \text{Var}[y_1] \left(\prod_{l=2}^L \frac{1}{2} n_l \text{Var}[w_l] \right). \quad (9)$$

这个乘积是初始化设计的核心所在。一个合理的初始化方法应当避免使输入信号的幅值呈指数级缩小或放大。因此,我们期望上述乘积能够取到一个合适的标量值(例如,1)。而满足这一要求的充分条件为:

$$\frac{1}{2} n_l \text{Var}[w_l] = 1, \quad \forall l. \quad (10)$$

这会形成均值为零的高斯分布,其标准差为 $\sqrt{2/n_l}$ 。这就是我们采用的初始化方式。同时,我们也会对 $\mathbf{b} = 0$ 进行初始化。

对于第一层($l = 1$),由于输入信号上没有应用ReLU激活函数,因此其方差应为 $n_1 \text{Var}[w_1] = 1$ 。不过,如果该因子仅出现在某一层中,其实并不会对结果产生影响。因此为了简化计算,我们在第一层也采用了公式(10)。

反向传播案例

在反向传播过程中,卷积层的梯度是通过以下方式计算得出的:

$$\Delta \mathbf{x}_l = \hat{W}_l \Delta \mathbf{y}_l. \quad (11)$$

为简化表达,此处我们使用 $\Delta \mathbf{x}$ 和 $\Delta \mathbf{y}$ 来表示梯度(即 $\frac{\partial \mathcal{E}}{\partial \mathbf{x}}$ 与 $\frac{\partial \mathcal{E}}{\partial \mathbf{y}}$)。其中 $\Delta \mathbf{y}$ 代表 d 个通道中 k 乘以 k 像素大小的数值。

该数值会被重新构建成一个 $k^2 d$ 乘以1的向量,我们将其记为 $\hat{n} = k^2 d$ 。需要注意的是, $\hat{n} \neq n = k^2 c$ 。 $\hat{W}$ 是一个 c 乘以 $\hat{n}$ 的矩阵,其中的滤波器是按照反向传播的顺序进行重新排列的。另外需注意, W 与 $\hat{W}$ 之间可以通过重新构形相互转换。 $\Delta \mathbf{x}$ 则是一个 c 乘以1的向量,用于表示该层中某个像素处的梯度。如前所述,假设 w_l 与 Δy_l 彼此独立,那么当 w_l 通过围绕零点的对称分布进行初始化时,所有 l 对应的 Δx_l 的均值均为零。

在反向传播中,我们还有 $\Delta y_l = f'(y_l) \Delta x_{l+1}$ 这一表达式,其中 f' 代表 f 的导数。对于ReLU情况而言, $f'(y_l)$ 的值要么为零,要么为一,且这两种情况的概率相等。我们假设 $f'(y_l)$ 与 Δx_{l+1} 彼此之间是独立的。由此可得 $E[\Delta y_l] = E[\Delta x_{l+1}]/2 = 0$,同时还有 $E[(\Delta y_l)^2] = \text{Var}[\Delta y_l] = \frac{1}{2} \text{Var}[\Delta x_{l+1}]$ 。随后,我们便能根据公式(11)计算出梯度的方差。

$$\begin{aligned} \text{Var}[\Delta x_l] &= \hat{n}_l \text{Var}[w_l] \text{Var}[\Delta y_l] \\ &= \frac{1}{2} \hat{n}_l \text{Var}[w_l] \text{Var}[\Delta x_{l+1}]. \end{aligned} \quad (12)$$

在公式(12)和公式(8)中出现的标量1/2都是ReLU激活函数运算的结果,只不过两者的推导过程有所不同。当将 L 层组合在一起时,我们可以得到:

$$\text{Var}[\Delta x_2] = \text{Var}[\Delta x_{L+1}] \left(\prod_{l=2}^L \frac{1}{2} \hat{n}_l \text{Var}[w_l] \right). \quad (13)$$

我们认为一个充分条件是梯度不会呈指数级地变大或变小。

$$\frac{1}{2} \hat{n}_l \text{Var}[w_l] = 1, \quad \forall l. \quad (14)$$

该方程与公式(10)的唯一区别在于,在 $n_l = k_l^2 c_l = k_l^2 d_{l-1}$ 期间值为 $\hat{n}_l = k_l^2 d_l$ 。而公式(14)会生成一个均值为零的高斯分布,其标准差为 $\sqrt{2/\hat{n}_l}$ 。

对于第一层($l = 1$)而言,由于它代表图像域,我们无需计算 Δx_1 。不过出于与前向传播情况相同的理由,我们仍可以在第一层使用公式(14)——单层的系数并不会使得整体乘积呈指数级地变大或变小。

需要指出的是,单独使用公式(14)或公式(10)中的任意一个都足够了。例如,若使用公式(14),则在公式(13)中乘积为 $\prod_{l=2}^L \frac{1}{2} \hat{n}_l \text{Var}[w_l] = 1$,在公式(9)中乘积为 $\prod_{l=2}^L \frac{1}{2} n_l \text{Var}[w_l] = \prod_{l=2}^L n_l / \hat{n}_l = c_2 / d_L$;而在常见的网络设计中,这些值并不会不断减小的数。这意味着,如果初始化能够合理调整反向传播的信号强度,那么正向传播的信号也会呈现相同的情况,反之亦然。在本文研究的所有模型中,这两种形式都能帮助模型收敛。

讨论

如果每层的正向/反向信号都被参数a进行不恰当的缩放,那么最终传递出的信号

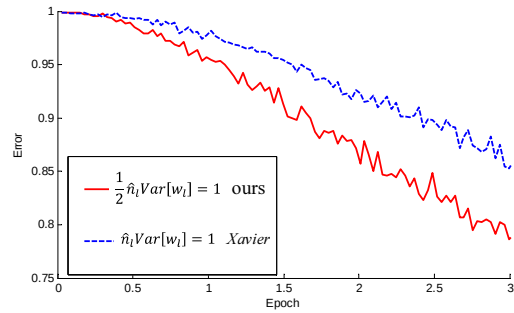


Figure 2. The convergence of a **22-layer** large model (B in Table 3). The x-axis is the number of training epochs. The y-axis is the top-1 error of 3,000 random val samples, evaluated on the center crop. We use ReLU as the activation for both cases. Both our initialization (red) and “Xavier” (blue) [7] lead to convergence, but ours starts reducing error earlier.

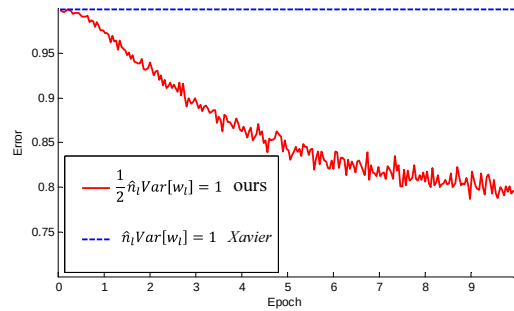


Figure 3. The convergence of a **30-layer** small model (see the main text). We use ReLU as the activation for both cases. Our initialization (red) is able to make it converge. But “Xavier” (blue) [7] completely stalls - we also verify that its gradients are all diminishing. It does not converge even given more epochs.

will be rescaled by a factor of β^L after L layers, where L can represent some or all layers. When L is large, if $\beta > 1$, this leads to extremely amplified signals and an algorithm output of infinity; if $\beta < 1$, this leads to diminishing signals². In either case, the algorithm does not converge - it diverges in the former case, and stalls in the latter.

Our derivation also explains why the constant standard deviation of 0.01 makes some deeper networks stall [25]. We take “model B” in the VGG team’s paper [25] as an example. This model has 10 conv layers all with 3×3 filters. The filter numbers (d_l) are 64 for the 1st and 2nd layers, 128 for the 3rd and 4th layers, 256 for the 5th and 6th layers, and 512 for the rest. The std computed by Eqn.(14) ($\sqrt{2/\hat{n}_l}$) is 0.059, 0.042, 0.029, and 0.021 when the filter numbers are 64, 128, 256, and 512 respectively. If the std is initialized

²In the presence of weight decay (l_2 regularization of weights), when the gradient contributed by the logistic loss function is diminishing, the total gradient is not diminishing because of the weight decay. A way of diagnosing diminishing gradients is to check whether the gradient is modulated only by weight decay.

as 0.01, the std of the gradient propagated from conv10 to conv2 is $1/(5.9 \times 4.2^2 \times 2.9^2 \times 2.1^4) = 1/(1.7 \times 10^4)$ of what we derive. This number may explain why diminishing gradients were observed in experiments.

It is also worth noticing that the variance of the input signal can be roughly preserved from the first layer to the last. In cases when the input signal is not normalized (*e.g.*, it is in the range of $[-128, 128]$), its magnitude can be so large that the softmax operator will overflow. A solution is to normalize the input signal, but this may impact other hyper-parameters. Another solution is to include a small factor on the weights among all or some layers, *e.g.*, $\sqrt[4]{1/128}$ on L layers. In practice, we use a std of 0.01 for the first two fc layers and 0.001 for the last. These numbers are smaller than they should be (*e.g.*, $\sqrt{2/4096}$) and will address the normalization issue of images whose range is about $[-128, 128]$.

For the initialization in the PReLU case, it is easy to show that Eqn.(10) becomes:

$$\frac{1}{2}(1 + a^2)n_l \text{Var}[w_l] = 1, \quad \forall l, \quad (15)$$

where a is the initialized value of the coefficients. If $a = 0$, it becomes the ReLU case; if $a = 1$, it becomes the linear case (the same as [7]). Similarly, Eqn.(14) becomes $\frac{1}{2}(1 + a^2)\hat{n}_l \text{Var}[w_l] = 1$.

Comparisons with “Xavier” Initialization [7]

The main difference between our derivation and the “Xavier” initialization [7] is that we address the rectifier nonlinearities³. The derivation in [7] only considers the linear case, and its result is given by $n_l \text{Var}[w_l] = 1$ (the forward case), which can be implemented as a zero-mean Gaussian distribution whose std is $\sqrt{1/n_l}$. When there are L layers, the std will be $1/\sqrt{2^L}$ of our derived std. This number, however, is not small enough to completely stall the convergence of the models actually used in our paper (Table 3, up to 22 layers) as shown by experiments. Figure 2 compares the convergence of a 22-layer model. Both methods are able to make them converge. But ours starts reducing error earlier. We also investigate the possible impact on accuracy. For the model in Table 2 (using ReLU), the “Xavier” initialization method leads to 33.90/13.44 top-1/top-5 error, and ours leads to 33.82/13.34. We have not observed clear superiority of one to the other on accuracy.

Next, we compare the two methods on extremely deep models with up to 30 layers (27 conv and 3 fc). We add up to sixteen conv layers with 256×2 filters in the model in

³There are other minor differences. In [7], the derived variance is adopted for uniform distributions, and the forward and backward cases are averaged. But it is straightforward to adopt their conclusion for Gaussian distributions and for the forward or backward case only.

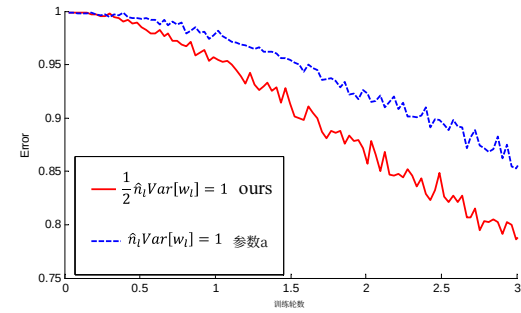


图2. 一个**22层**大型模型（见表3中的B）的收敛情况。x轴表示训练轮数，y轴则表示在中心区域裁剪得到的3,000个随机验证样本的Top-1误差。两种情况下我们都采用了ReLU作为激活函数。无论是我们的初始化方法（红色曲线）还是“Xavier”初始化方法（蓝色曲线）[7]，都能使模型收敛，不过采用我们自己的初始化方法能更早开始降低误差。

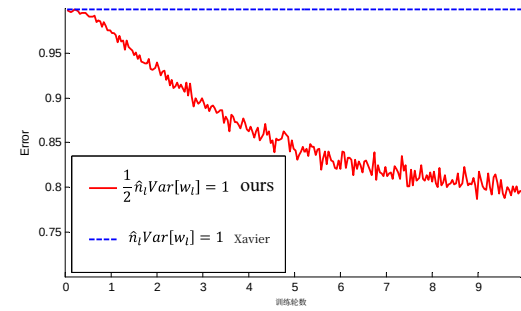


图3. 一个**30层**小型模型的收敛情况（详见正文）。两种情况下同样使用ReLU作为激活函数。我们的初始化方法（红色曲线）能够促使模型收敛，而“Xavier”初始化方法（蓝色曲线）[7]则完全无法让模型收敛——我们还发现其梯度一直在持续减小。即便增加训练轮数，该模型依然无法收敛。

在经过b参数个层之后会再次被参数a缩放一次，其中b参数可以代表部分层或所有层。当该参数值较大时，若满足特定条件，就会导致信号被极度放大，从而使算法输出无穷大；而若不满该条件，则会导致信号逐渐衰减。无论哪种情况，算法都无法收敛——前者会出现发散现象，后者则会停滞不前。

我们的推导过程还解释了为何恒定的标准差为0.01会导致某些更深层次的网络无法继续运行 [25]。我们以VGG团队的论文 [25] 中提到的“模型B”为例：该模型包含10个卷积层，每个层都配有 3×3 个滤波器。这些层的滤波器数量分别为第1层和第2层为64个，第3层和第4层为128个，第5层和第6层为256个，其余层则为512个。根据公式(14) ($\sqrt{2/\hat{n}_l}$) 计算得出的标准差在滤波器数量为64、128、256和512时分别对应 0.059、0.042、0.029和0.021。如果将标准差初始化为0.01，那么从 conv10传递到conv2的梯度标准差将为我们推导结果的 [25] ([25]) 3×3 (d_l)倍，这一数值或许可以解释实验中观察到的梯度逐渐减小的现象。

²在存在权重衰减（即对权重进行的正则化处理）的情况下，虽然逻辑损失函数产生的梯度在不断减小，但由于权重衰减的存在，总梯度并不会随之减小。判断梯度是否正在减小的一个方法是检查该梯度是否仅受到权重衰减的影响。

也就是说，当标准差被初始化为0.01时，从conv10传递到conv2的梯度标准差将是我们推导结果对应的 $1/(5.9 \times 4.2^2 \times 2.9^2 \times 2.1^4) = 1/(1.7 \times 10^4)$ 倍，这也许就是实验中出现梯度逐渐减小的原因。

另外值得注意的是，输入信号的方差大致可以从第一层保持到最后一层。在输入信号未经过归一化处理（例如，其取值范围为 $[-128, 128]$ ）的情况下，其幅值可能会大到导致softmax运算发生溢出。一种解决方案是对输入信号进行归一化，但这可能会影响其他超参数。另一种方法是在全部或部分层的权重上乘以一个较小的系数，例如，在 L 层上使用 $\sqrt[4]{1/128}$ 作为该系数。在实际应用中，我们对前两层全连接层采用标准差0.01，而对最后一层则采用0.001。这些数值虽然比理想值要小（例如， $\sqrt{2/4096}$ ），但足以解决那些取值范围约为 $[-128, 128]$ 的图像所面临的归一化问题。

对于PReLU模型中的初始化问题，很容易证明公式 (10)会变为：

$$\frac{1}{2}(1 + a^2)n_l \text{Var}[w_l] = 1, \quad \forall l, \quad (15)$$

其中 a 即为系数的初始值。当取 $a = 0$ 时，对应ReLU激活函数情形；而当取 $a = 1$ 时，则为线性情形（与 [7] 14 $\frac{1}{2} 1 +$ 相同）。同样地， a^2 方程()也会变为 $(\hat{})n_l \text{Var}[w_l] = 1$ 。

与 “Xavier” 初始化方法的对比 [7]

我们的推导结果与“Xavier”初始化方法 [7] 的主要区别在于，我们考虑了整流非线性单元³的影响。[7] 中的推导仅针对线性情形，其结果由 $n_l \text{Var}[w_l] = 1$ （即前向情形）给出，该结果可表示为标准差为 $\sqrt{1/n_l}$ 的零均值高斯分布。当模型包含 L 层时，其标准差将为我们所推导出的标准差的 $1/\sqrt{2^L}$ 倍。然而实验表明，这一数值并不足以完全阻碍本文实际使用的模型（表3中最多22层模型）的收敛过程。图2展示了22层模型的收敛情况，两种方法都能使模型收敛，但我们的方法更早开始降低误差。我们还研究了其对准确率可能产生的影响：对于表2中采用ReLU激活函数的模型，“Xavier”初始化方法对应的Top-1和Top-5错误率分别为33.90/13.44，而我们的方法对应的数值则为33.82/13.34。总体而言，我们并未观察到两种方法在准确率方面存在明显的优劣差异。

接下来，我们将在深度极深的模型上对比这两种方法，这类模型的层数可达30层（包含27个卷积层和3个全连接层）。我们在模型中额外增加了多达16个卷积层，每个层配备 256×2 个滤波器。

³除此之外还有其他一些细微差别。在 [7] 中，对于均匀分布会采用推导出的方差值，并对前向情形与后向情形的结果取平均值；而对于高斯分布，或是仅针对前向情形、仅针对后向情形而言，直接采用相关结论即可。

Table 1. Figure 3 shows the convergence of the 30-layer model. Our initialization is able to make the extremely deep model converge. On the contrary, the “*Xavier*” method completely stalls the learning, and the gradients are diminishing as monitored in the experiments.

These studies demonstrate that we are ready to investigate extremely deep, rectified models by using a more principled initialization method. But in our current experiments on ImageNet, we have not observed the benefit from training extremely deep models. For example, the aforementioned 30-layer model has 38.56/16.59 top-1/top-5 error, which is clearly worse than the error of the 14-layer model in Table 2 (33.82/13.34). Accuracy saturation or degradation was also observed in the study of small models [10], VGG’s large models [25], and in speech recognition [34]. This is perhaps because the method of increasing depth is not appropriate, or the recognition task is not enough complex.

Though our attempts of extremely deep models have not shown benefits, our initialization method paves a foundation for further study on increasing depth. We hope this will be helpful in other more complex tasks.

2.3. Architectures

The above investigations provide guidelines of designing our architectures, introduced as follows.

Our baseline is the 19-layer model (A) in Table 3. For a better comparison, we also list the VGG-19 model [25]. Our model A has the following modifications on VGG-19: (i) in the first layer, we use a filter size of 7×7 and a stride of 2; (ii) we move the other three conv layers on the two largest feature maps (224, 112) to the smaller feature maps (56, 28, 14). The time complexity (Table 3, last row) is roughly unchanged because the deeper layers have more filters; (iii) we use spatial pyramid pooling (SPP) [11] before the first fc layer. The pyramid has 4 levels - the numbers of bins are 7×7 , 3×3 , 2×2 , and 1×1 , for a total of 63 bins.

It is worth noticing that we have no evidence that our model A is a better *architecture* than VGG-19, though our model A has better results than VGG-19’s result reported by [25]. In our earlier experiments with less scale augmentation, we observed that our model A and our reproduced VGG-19 (with SPP and our initialization) are comparable. The main purpose of using model A is for faster running speed. The actual running time of the conv layers on larger feature maps is slower than those on smaller feature maps, when their time complexity is the same. In our four-GPU implementation, our model A takes 2.6s per mini-batch (128), and our reproduced VGG-19 takes 3.0s, evaluated on four Nvidia K20 GPUs.

In Table 3, our model B is a deeper version of A. It has three extra conv layers. Our model C is a wider (with more filters) version of B. The width substantially increases the

complexity, and its time complexity is about $2.3\times$ of B (Table 3, last row). Training A/B on four K20 GPUs, or training C on eight K40 GPUs, takes about 3-4 weeks.

We choose to increase the model width instead of depth, because deeper models have only diminishing improvement or even degradation on accuracy. In recent experiments on small models [10], it has been found that aggressively increasing the depth leads to saturated or degraded accuracy. In the VGG paper [25], the 16-layer and 19-layer models perform comparably. In the speech recognition research of [34], the deep models degrade when using more than 8 hidden layers (all being fc). We conjecture that similar degradation may also happen on larger models for ImageNet. We have monitored the training procedures of some extremely deep models (with 3 to 9 layers added on B in Table 3), and found both training and testing error rates degraded in the first 20 epochs (but we did not run to the end due to limited time budget, so there is not yet solid evidence that these large and overly deep models will ultimately degrade). Because of the possible degradation, we choose not to further increase the depth of these large models.

On the other hand, the recent research [5] on small datasets suggests that the accuracy should improve from the increased number of parameters in conv layers. This number depends on the depth and width. So we choose to increase the width of the conv layers to obtain a higher-capacity model.

While all models in Table 3 are very large, we have not observed severe overfitting. We attribute this to the aggressive data augmentation used throughout the whole training procedure, as introduced below.

3. Implementation Details

Training

Our training algorithm mostly follows [16, 13, 2, 11, 25]. From a resized image whose shorter side is s , a 224×224 crop is randomly sampled, with the per-pixel mean subtracted. The scale s is randomly jittered in the range of [256, 512], following [25]. One half of the random samples are flipped horizontally [16]. Random color altering [16] is also used.

Unlike [25] that applies scale jittering only during fine-tuning, we apply it from the beginning of training. Further, unlike [25] that initializes a deeper model using a shallower one, we directly train the very deep model using our initialization described in Sec. 2.2 (we use Eqn.(14)). Our end-to-end training may help improve accuracy, because it may avoid poorer local optima.

Other hyper-parameters that might be important are as follows. The weight decay is 0.0005, and momentum is 0.9. Dropout (50%) is used in the first two fc layers. The mini-batch size is fixed as 128. The learning rate is $1e-2$, $1e-3$,

表1及图3展示了30层模型的收敛情况。我们所采用的初始化方法能够让深度极深的模型实现收敛；而相反，采用“*Xavier*”初始化方法则会导致学习过程完全停滞，实验数据也显示其梯度值持续下降。

这些研究表明，通过采用更为合理的初始化方法，我们已具备研究深度极深的整流化模型的条件。不过在目前针对ImageNet数据集的实验中，我们并未观察到训练深度极深模型所带来的优势。例如，前述的30层模型的Top-1和Top-5错误率分别为38.56/16.59，这一数值明显高于表2中14层模型的错误率（33.82/13.34）。在针对小型模型 [10],VGG的网络大型版本 [25], 以及语音识别领域 [34]的研究中，也观察到了准确率饱和或下降的现象。出现这种情况的原因或许在于增加模型深度的策略并不合适，或是相应的识别任务复杂度尚不足。

尽管我们尝试构建深度极深的模型并未取得预期效果，但所采用的初始化方法仍为后续深入研究如何进一步提升模型深度奠定了基础。我们希望这一成果能对其他更为复杂的任务研究有所帮助。

2.3. 架构设计

上述研究为我们的架构设计提供了指导原则，具体内容如下。

我们的基准模型是表3中的19层模型（A）。为便于对比，我们还列出了VGG-19模型 [25]。模型A在VGG-19的基础上进行了以下改进：(i) 在第一层，我们采用 7×7 的滤波器尺寸和步长为2的设置；(ii) 将另外三个卷积层从最大的特征图（224、112）移至较小的特征图（56、28、14）上。由于深层包含的滤波器数量更多，其时间复杂度（见表3最后一行）基本保持不变；(iii) 我们在第一个全连接层之前加入了空间金字塔池化算法（SPP） [11]。该金字塔结构共有4个层级，对应的箱数分别为 7×7 、 3×3 、 2×2 和 1×1 ，总计63个箱。

值得注意的是，尽管我们的模型A的测试准确率优于 [25]所报告的VGG-19的结果，但我们并没有证据表明模型A在架构层面优于VGG-19。在之前采用较少尺度增强处理的实验中，我们发现模型A与经过我们重新实现的、应用了空间金字塔池化算法及特定初始化方式的VGG-19性能相当。采用模型A的主要目的在于提升运行速度——在时间复杂度相同的情况下，处理更大尺寸特征图时的卷积层实际运行速度会慢于处理较小特征图时的速度。在我们的四块K20型GPU配置下，模型A处理每个包含128个样本的迷你批次需要2.6秒，而经过重新实现的VGG-19则需3.0秒。

在表3中，我们的模型B是模型A的深度更高版本，它多了三层额外卷积层。而模型C则是模型B的宽度更大（滤波器数量更多）的版本，这种更高的模型宽度会显著提升其复杂度，其时间复杂度约为模型B的 $2.\{v\}$ 倍（见表3的最后一行）。在四颗K20型GPU上对模型A或B进行训练，或在八颗K40型GPU上对模型C进行训练，大约需要3到4周的时间。

上述训练过程的时间复杂度较高，相关数据可见表3的最后一行。

我们选择增加模型的宽度而非深度，因为深度更高的模型在准确率方面的提升幅度越来越小，甚至可能出现下降。在针对小型模型的最新实验 [10], 中，人们发现过度增加模型深度会导致准确率达到饱和或出现下降。在VGG论文 [25], 中，16层模型与19层模型的表现相当。而在[34], 的语音识别研究里，深度超过8层（且所有层均为全连接层）的模型性能也会下降。我们推测，在用于ImageNet任务的更大规模模型上也可能出现类似的性能退化现象。我们监测了表3中B类那些额外增加了3到9层结构的深度极深模型的训练过程，发现在前20个训练轮数内，这些模型的训练误差率和测试误差率都会上升（但由于时间限制并未将训练进行到底，因此目前还无法确定这类庞大且深度过高的模型最终一定会出现性能退化）。正因存在这种潜在的退化风险，我们决定不再进一步增加这些大型模型的深度。

另一方面，近期针对小规模数据集的 [5]研究表明，由于卷积层中参数数量的增加，模型的准确率应当会有所提升。这一参数数量取决于层的深度与模型宽度，因此我们选择增大卷积层的模型宽度，以此构建出容量更高的模型。

尽管表3中的所有模型规模都相当庞大，但我们并未观察到严重的过拟合现象。我们认为这是由于在整个训练过程中采用了较为激进的数据增强策略，具体内容将在下文介绍。

3. 实现细节

训练

我们的训练算法在很大程度上遵循了 [16, 13, 2, 11, 25]的方法。首先从短边长度为 s 的调整后图像中随机选取一个 224×224 大小的区域，然后从中减去每个像素的均值。随后，会按照 [25]的规则在[256, 512], 的范围内对缩放比例 s 进行随机抖动处理。此外，一半的随机样本还会被水平翻转 [16], 同时也会运用随机颜色修改 [16] 的技术。

与仅在微调阶段才应用缩放抖动的 [25] 不同，我们会从训练开始就一直使用这种技术。再者，与通过较浅的模型来初始化更深层次模型的 [25] 不同，我们会直接利用第2.2节中描述的初始化方法（即公式(14)）来训练那些非常深的模型。这种端到端的训练方式有助于提升模型的准确率，因为它能够避免陷入质量较差的局部最优解。

其他可能重要的超参数如下：权重衰减值为0.0005，动量为0.9；前两个全连接层采用了50%的Dropout机制。小批量大小固定为128，学习率则分别为 $1e-2$ 和 $1e-3$ ，当误差趋于稳定时还会切换使用 $1e-4$ 的参数值。每个模型的训练轮数大约为80轮。

input size	VGG-19 [25]	model A	model B	model C
224	3×3, 64 3×3, 64 2×2 maxpool, /2	7×7, 96, /2	7×7, 96, /2	7×7, 96, /2
112	3×3, 128 3×3, 128 2×2 maxpool, /2			
56	3×3, 256 3×3, 256 3×3, 256 3×3, 256 2×2 maxpool, /2	3×3, 256 3×3, 256 3×3, 256 3×3, 256 3×3, 256 2×2 maxpool, /2	3×3, 256 3×3, 256 3×3, 256 3×3, 256 3×3, 256 2×2 maxpool, /2	3×3, 384 3×3, 384 3×3, 384 3×3, 384 3×3, 384 2×2 maxpool, /2
28	3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2 maxpool, /2	3×3, 512 3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2 maxpool, /2	3×3, 512 3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2 maxpool, /2	3×3, 768 3×3, 768 3×3, 768 3×3, 768 3×3, 768 2×2 maxpool, /2
14	3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2 maxpool, /2	3×3, 512 3×3, 512 3×3, 512 3×3, 512 spp, {7, 3, 2, 1}	3×3, 512 3×3, 512 3×3, 512 3×3, 512 spp, {7, 3, 2, 1}	3×3, 896 3×3, 896 3×3, 896 3×3, 896 spp, {7, 3, 2, 1}
fc ₁ fc ₂ fc ₃	4096 4096 1000			
depth (conv+fc)	19	19	22	22
complexity (ops., ×10 ¹⁰)	1.96	1.90	2.32	5.30

Table 3. Architectures of large models. Here “/2” denotes a stride of 2.

and 1e-4, and is switched when the error plateaus. The total number of epochs is about 80 for each model.

Testing

We adopt the strategy of “multi-view testing on feature maps” used in the SPP-net paper [11]. We further improve this strategy using the dense sliding window method in [24, 25].

We first apply the convolutional layers on the resized full image and obtain the last convolutional feature map. In the feature map, each 14×14 window is pooled using the SPP layer [11]. The fc layers are then applied on the pooled features to compute the scores. This is also done on the horizontally flipped images. The scores of all dense sliding windows are averaged [24, 25]. We further combine the results at multiple scales as in [11].

Multi-GPU Implementation

We adopt a simple variant of Krizhevsky’s method [15] for parallel training on multiple GPUs. We adopt “data parallelism” [15] on the conv layers. The GPUs are synchronized before the first fc layer. Then the forward/backward propagations of the fc layers are performed on a single GPU - this means that we do not parallelize the computation of the fc layers. The time cost of the fc layers is low, so it is not necessary to parallelize them. This leads to a simpler implementation than the “model parallelism” in [15]. Besides, model parallelism introduces some overhead due to the communication of filter responses, and is not faster than computing the fc layers on just a single GPU.

We implement the above algorithm on our modification of the Caffe library [14]. We do not increase the mini-batch size (128) because the accuracy may be decreased [15]. For the large models in this paper, we have observed a 3.8x speedup using 4 GPUs, and a 6.0x speedup using 8 GPUs.

输入规模	VGG-19 [25]	模型A	模型B	模型C
224	3×3, 64 3×3, 64 2×2最大池化, 步长为/2	7×7, 96, /2	7×7, 96, /2	7×7, 96, /2
112	3×3, 128 3×3, 128 2×2参数a maxpool, 缩放比例为1/2			2×2 参数a maxpool, 缩放比例为1/2
56	3×3, 256 3×3, 256 3×3, 256 3×3, 256 2×2参数a maxpool, 缩放比例为1/2	3×3, 256 3×3, 256 3×3, 256 3×3, 256 3×3, 256 2×2 参数a maxpool, 缩放比例为1/2	3×3, 256 3×3, 256 3×3, 256 3×3, 256 3×3, 256 2×2 参数a maxpool, 缩放比例为2	3×3, 384 3×3, 384 3×3, 384 3×3, 384 3×3, 384 2×2 参数a maxpool, 缩放比例为1/2
28	3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2参数a maxpool, 缩放比例为/2	3×3, 512 3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2 参数a maxpool, 缩放比例为/2	3×3, 512 3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2 参数a maxpool, 缩放比例为/2	3×3, 768 3×3, 768 3×3, 768 3×3, 768 3×3, 768 2×2 参数a maxpool, 缩放比例为/2
14	3×3, 512 3×3, 512 3×3, 512 3×3, 512 2×2最大池化操作, 池化步长为/2	3×3, 512 3×3, 512 3×3, 512 3×3, 512 空间金字塔池化算法, 参数为 {7, 3, 2, 1}	3×3, 512 3×3, 512 3×3, 512 3×3, 512 3×3, 512 空间金字塔池化算法, 参数为 {7, 3, 2, 1}	3×3, 896 3×3, 896 3×3, 896 3×3, 896 3×3, 896 空间金字塔池化算法, 参数为 {7, 3, 2, 1}
fc ₁ fc ₂ fc ₃	4096 4096 1000			
网络深度（由conv+fc层构成）	19	19	22	22
计算复杂度（操作次数为 ×10 ¹⁰ ）	1.96	1.90	2.32	5.30

表3. 大型模型的架构。此处 “/2” 表示步长为2。

此外，当误差趋于稳定时还会启用1e-4的参数设置。每个模型的总训练轮数约为80轮。

测试

我们采用了SPP-net相关论文 [11]中提出的 “基于特征图的多视图测试” 策略，并进一步通过 [24, 25]中的密集滑动窗口法对该策略进行了优化。

首先，我们会在调整尺寸后的完整图像上应用卷积层，从而得到最终的卷积特征图。在该特征图中，每个 14×14窗口都会通过SPP层 [11]进行池化处理。随后，全连接层会被应用于这些经过池化处理后的特征，以此计算出对应的得分。同样的操作也会在水平翻转后的图像上进行。所有密集排列的滑动窗口所对应的得分会被取平均值 [24, 25]。最后，我们会像 [11]中描述的那样，将不同尺度下的结果进行整合。

多图形处理器实现

为了在多台图形处理器上实现并行训练，我们采用了Krizhevsky方法的简化版本 [15]。在卷积层上，我们采用了 “数据并行” 策略 [15]。在第一个全连接层之前，这些图形处理器会先完成同步。之后，全连接层的正向/反向传播运算则会在单台图形处理器上执行——也就是说，我们并没有对全连接层的计算过程进行并行化处理。由于全连接层的计算耗时较短，因此没有必要对其进行并行化。这样一来，其实现方式就比 [15]中提到的 “模型并行” 更为简单。此外，模型并行由于需要传递滤波器响应信息，会带来一定的额外开销，其速度也不比仅在单台图形处理器上计算全连接层更快。

我们在对Caffe库 [14]进行的改进版本上实现了上述算法。由于增加小批量大小（128）可能会导致准确率下降 [15]，因此我们并未调整该数值。对于本文中涉及的大型模型，使用4块图形处理器时性能可提升3.8倍，而使用8块图形处理器时则可实现6.0倍的加速。

model A	ReLU		PReLU	
scale s	top-1	top-5	top-1	top-5
256	26.25	8.25	25.81	8.08
384	24.77	7.26	24.20	7.03
480	25.46	7.63	24.83	7.39
multi-scale	24.02	6.51	22.97	6.28

Table 4. Comparisons between ReLU/PReLU on model A in ImageNet 2012 using dense testing.

4. Experiments on ImageNet

We perform the experiments on the 1000-class ImageNet 2012 dataset [22] which contains about 1.2 million training images, 50,000 validation images, and 100,000 test images (with no published labels). The results are measured by top-1/top-5 error rates [22]. We only use the provided data for training. All results are evaluated on the validation set, except for the final results in Table 7, which are evaluated on the test set. The top-5 error rate is the metric officially used to rank the methods in the classification challenge [22].

Comparisons between ReLU and PReLU

In Table 4, we compare ReLU and PReLU on the large model A. We use the channel-wise version of PReLU. For fair comparisons, both ReLU/PReLU models are trained using the same total number of epochs, and the learning rates are also switched after running the same number of epochs.

Table 4 shows the results at three scales and the multi-scale combination. The best single scale is 384, possibly because it is in the middle of the jittering range [256, 512]. For the multi-scale combination, PReLU reduces the top-1 error by 1.05% and the top-5 error by 0.23% compared with ReLU. The results in Table 2 and Table 4 consistently show that PReLU improves both small and large models. This improvement is obtained with almost no computational cost.

Comparisons of Single-model Results

Next we compare single-model results. We first show 10-view testing results [16] in Table 5. Here, each view is a 224-crop. The 10-view results of VGG-16 are based on our testing using the publicly released model [25] as it is not reported in [25]. Our best 10-view result is 7.38% (Table 5). Our other models also outperform the existing results.

Table 6 shows the comparisons of single-model results, which are all obtained using multi-scale and multi-view (or dense) test. Our results are denoted as MSRA. Our baseline model (A+ReLU, 6.51%) is already substantially better than the best existing single-model result of 7.1% reported for VGG-19 in the latest update of [25] (arXiv v5). We be-

lieve that this gain is mainly due to our end-to-end training, without the need of pre-training shallow models.

Moreover, our best single model (C, PReLU) has **5.71%** top-5 error. This result is even better than all previous multi-model results (Table 7). Comparing A+PReLU with B+PReLU, we see that the 19-layer model and the 22-layer model perform comparably. On the other hand, increasing the width (C vs. B, Table 6) can still improve accuracy. This indicates that when the models are deep enough, the width becomes an essential factor for accuracy.

Comparisons of Multi-model Results

We combine six models including those in Table 6. For the time being we have trained only one model with architecture C. The other models have accuracy inferior to C by considerable margins. We conjecture that we can obtain better results by using fewer stronger models.

The multi-model results are in Table 7. Our result is **4.94%** top-5 error on the test set. This number is evaluated by the ILSVRC server, because the labels of the test set are not published. Our result is 1.7% better than the ILSVRC 2014 winner (GoogLeNet, 6.66% [29]), which represents a $\sim 26\%$ relative improvement. This is also a $\sim 17\%$ relative improvement over the latest result (Baidu, 5.98% [32]).

Analysis of Results

Figure 4 shows some example validation images successfully classified by our method. Besides the correctly predicted labels, we also pay attention to the other four predictions in the top-5 results. Some of these four labels are other objects in the multi-object images, *e.g.*, the “horse-cart” image (Figure 4, row 1, col 1) contains a “mini-bus” and it is also recognized by the algorithm. Some of these four labels are due to the uncertainty among similar classes, *e.g.*, the “coucal” image (Figure 4, row 2, col 1) has predicted labels of other bird species.

Figure 6 shows the per-class top-5 error of our result (average of 4.94%) on the test set, displayed in ascending order. Our result has zero top-5 error in 113 classes - the images in these classes are all correctly classified. The three classes with the highest top-5 error are “letter opener” (49%), “spotlight” (38%), and “restaurant” (36%). The error is due to the existence of multiple objects, small objects, or large intra-class variance. Figure 5 shows some example images misclassified by our method in these three classes. Some of the predicted labels still make some sense.

In Figure 7, we show the per-class difference of top-5 error rates between our result (average of 4.94%) and our team’s in-competition result in ILSVRC 2014 (average of 8.06%). The error rates are reduced in 824 classes, unchanged in 127 classes, and increased in 49 classes.

模型A	ReLU		PReLU	
缩放参数 s	Top-1错误率	Top-5错误率	Top-1错误率	Top-5错误率
256	26.25	8.25	25.81	8.08
384	24.77	7.26	24.20	7.03
480	25.46	7.63	24.83	7.39
多尺度	24.02	6.51	22.97	6.28

表4：在ImageNet 2012数据集上使用密集测试对大型模型A中的ReLU与PReLU进行对比。

4. ImageNet数据集上的实验

实验是在包含约120万张训练图像、5万张验证图像以及10万张测试图像（这些测试图像没有公开的标签）的1000类ImageNet 2012数据集 [22] 上开展的。实验结果通过Top-1和Top-5错误率 [22]来衡量。我们仅使用所提供的数据来进行模型训练，所有结果均在验证集上评估，表7中的最终结果除外，后者是在测试集上评估得出的。在本次分类挑战赛中，Top-5错误率是用于对各种方法进行排名的官方标准指标 [22]。

ReLU与PReLU的对比

在表4中，我们对大型模型A上的ReLU与PReLU进行了比较。此处采用的是PReLU的通道级版本。为确保对比的公平性，两种ReLU及PReLU模型均使用相同总数的训练轮数进行训练，且在完成相同数量的训练轮数后也会更换学习率。

表4展示了三种不同尺度下的测试结果以及多尺度组合的测试结果。其中最佳的单尺度为384，这很可能是因为该数值处于波动范围 [256, 512]的中间位置。在多尺度组合的测试中，与ReLU激活函数相比，PReLU能够将Top-1错误率降低1.05%，同时将前五名错误率降低0.23%。表2和表4中的结果均表明，PReLU能够提升小型和大型模型的性能，而且几乎不会带来额外的计算开销。

单模型结果对比

接下来我们将对单模型的测试结果进行对比。首先，表5展示了基于10种视图方式的测试结果 [16]，这里的每种视图对应一次224像素大小的图像裁剪。VGG-16在10种视图方式下的测试结果是基于我们使用公开发布的模型 [25] 所获得的，因为相关数据并未在 [25]中给出。我们取得的最佳10视图测试结果为7.38%（见表5），其他模型也同样优于现有的同类成果。

表6展示了各类单模型测试结果的对比情况，这些结果均是通过对多尺度及多视角（或密集）测试获得的。我们的实验结果以MSRA作为标识。我们所使用的基线模型（A+ReLU，准确率为6.51%）已经显著优于 [25]（arXiv v5）最新版本中针对VGG-19报告的7.1%这一最佳现有单模型测试结果。我们认

为这种优势主要源于我们所采用的端到端训练方式，无需对浅层模型进行预训练。

此外，我们表现最佳的单一模型（C结构搭配PReLU激活函数）的前五名错误率为**5.71%**。这一成绩甚至优于此前所有的多模型组合结果（见表7）。将A+PReLU与B+PReLU进行对比后可以发现，19层模型与22层模型的性能相当。另一方面，增加模型宽度（参见C与B的对比，见表6）仍能进一步提升准确率，这表明当模型深度足够大时，模型宽度就会成为决定准确率的关键因素。

多模型结果对比

我们共整合了包括表6中所列模型在内的六种模型。目前仅训练了采用C结构的一种模型，其余模型的准确率都比C模型低很多。我们认为，使用数量更少但性能更强的模型，有望取得更好的成绩。

多模型测试的结果展示在表7中。在我们的测试集上，该模型的前五名错误率为**4.94%**。由于测试集的标签并未公开，这一数值是由ILSVRC评测服务器评估得出的。我们的成绩比2014年ILSVRC竞赛的获胜者（GoogLeNet模型，错误率为6.66% [29]）好1.7%，相当于相对提升了 $\sim 26\%$ ；同时相较于最新的成果（百度模型的错误率为5.98% [32]），也实现了 $\sim 17\%$ 的相对提升。

结果分析

图4展示了通过该方法成功分类的若干验证图像示例。除了正确预测的标签外，我们还会关注Top-5结果中的其他四个预测值。这四个标签中有一些是多目标图像中的其他物体，例如，图4第1行第1列的“马车”图像中就包含了一辆“小型巴士”，该物体也被算法识别了出来。另外一些标签则源于相似类别之间的不确定性，例如，图4第2行第1列的“杜鹃鸟”图像所对应的预测标签就是其他鸟类物种。

图6按升序展示了该方法在测试集上的各类别Top-5错误率情况（平均值为4.94%）。在该方法的处理下，有113个类别的Top-5错误率为零，即这些类别中的图像都得到了正确分类。Top-5错误率最高的三个类别分别是“开信刀”（49%）、“聚光灯”（38%）和“餐厅”（36%）。出现这类错误的原因包括存在多个物体、物体尺寸过小，或是类内方差过大。图5则展示了这三个类别中因该方法而出现分类错误的若干图像示例，其中部分预测标签仍具有一定的合理性。

如图7所示，我们展示了在我们的结果（平均Top-5错误率为4.94%）与团队在2014年ILSVRC竞赛中的实际成绩（平均Top-5错误率为8.06%）之间，各类别的Top-5错误率差异。其中824个类别的错误率有所下降，127个类别的错误率保持不变，还有49个类别的错误率上升。

model	top-1	top-5
MSRA [11]	29.68	10.95
VGG-16 [25]	28.07 [†]	9.33 [†]
GoogLeNet [29]	-	9.15
A, ReLU	26.48	8.59
A, PReLU	25.59	8.23
B, PReLU	25.53	8.13
C, PReLU	24.27	7.38

Table 5. The single-model **10-view** results for ImageNet 2012 val set. [†]: Based on our tests.

	team	top-1	top-5
in competition ILSVRC 14	MSRA [11]	27.86	9.08 [†]
	VGG [25]	-	8.43 [†]
	GoogLeNet [29]	-	7.89
post-competition	VGG [25] (arXiv v2)	24.8	7.5
	VGG [25] (arXiv v5)	24.4	7.1
	Baidu [32]	24.88	7.42
	MSRA (A, ReLU)	24.02	6.51
	MSRA (A, PReLU)	22.97	6.28
	MSRA (B, PReLU)	22.85	6.27
	MSRA (C, PReLU)	21.59	5.71

Table 6. The **single-model** results for ImageNet 2012 val set. [†]: Evaluated from the test set.

	team	top-5 (test)
in competition ILSVRC 14	MSRA, SPP-nets [11]	8.06
	VGG [25]	7.32
	GoogLeNet [29]	6.66
post-competition	VGG [25] (arXiv v5)	6.8
	Baidu [32]	5.98
	MSRA, PReLU-nets	4.94

Table 7. The **multi-model** results for the ImageNet 2012 test set.

Comparisons with Human Performance from [22]

Russakovsky *et al.* [22] recently reported that human performance yields a 5.1% top-5 error on the ImageNet dataset. This number is achieved by a human annotator who is well trained on the validation images to be better aware of the existence of relevant classes. When annotating the test images, the human annotator is given a special interface, where each class title is accompanied by a row of 13 example training images. The reported human performance is estimated on a random subset of 1500 test images.

Our result (4.94%) exceeds the reported human-level performance. To our knowledge, our result is the first published instance of surpassing humans on this visual recognition challenge. The analysis in [22] reveals that the two major types of human errors come from fine-grained recognition and class unawareness. The investigation in [22] sug-

gests that algorithms can do a better job on fine-grained recognition (*e.g.*, 120 species of dogs in the dataset). The second row of Figure 4 shows some example fine-grained objects successfully recognized by our method - “coucal”, “komondor”, and “yellow lady’s slipper”. While humans can easily recognize these objects as a bird, a dog, and a flower, it is nontrivial for most humans to tell their species. On the negative side, our algorithm still makes mistakes in cases that are not difficult for humans, especially for those requiring context understanding or high-level knowledge (*e.g.*, the “spotlight” images in Figure 5).

While our algorithm produces a superior result on this particular dataset, this does not indicate that machine vision outperforms human vision on object recognition in general. On recognizing elementary object categories (*i.e.*, common objects or concepts in daily lives) such as the Pascal VOC

模型	Top-1错误率	Top-5错误率
微软研究院 [11]	29.68	10.95
VGG-16 [25]	28.07 [†]	9.33 [†]
GoogLeNet [29]	-	9.15
A, ReLU	26.48	8.59
A, PReLU	25.59	8.23
B, PReLU	25.53	8.13
C, PReLU	24.27	7.38

表5. ImageNet 2012验证集上基于单模型的**10种视角**测试结果 [†]: 数据来源于我们的测试。

	team	Top-1错误率	Top-5错误率
正在参与竞赛 ILSVRC 2014	微软研究院 [11]	27.86	9.08[†]
	VGG [25]	-	8.43 [†]
	GoogLeNet [29]	-	7.89
竞赛结束后	VGG [25] （arXiv v2版本）	24.8	7.5
	VGG [25] （arXiv v5版本）	24.4	7.1
	百度 [32]	24.88	7.42
	微软研究院（A, ReLU激活函数）	24.02	6.51
	微软研究院（A, PReLU）	22.97	6.28
	微软研究院（B, PReLU）	22.85	6.27
	微软研究院（C, PReLU）	21.59	5.71

表6。单模型在ImageNet 2012验证集上的测试结果 [†]: 评估数据来自测试集。

	team	Top-5 (测试)
正在参与竞赛 ILSVRC 2014	微软研究院与SPP网络 [11]	8.06
	VGG网络 [25]	7.32
	GoogLeNet模型 [29]	6.66
竞赛结束后	VGG [25] (arXiv v5版本)	6.8
	百度 [32]	5.98
	MSRA, PReLU网络	4.94

表7。ImageNet 2012测试集的**多模型**评测结果。

与 [22]中记载的人类表现对比

Russakovsky 等人在 [22]最近的研究中指出，人类在 ImageNet数据集上的人类表现对应的Top-5错误率为5.1%。这一数值是由一位经过专业训练、对验证图像极为熟悉的标注员所取得的，他能够更准确地识别出相关类别。在标注测试图像时，该标注员会使用一个专门的界面，每个类别名称旁都会展示一行共13张训练样本图像。上述人类表现的数据是基于1500张测试图像的随机子集估算得出的。

我们的结果（4.94%）超过了已有文献中记载的人类水平性能。据我们所知，这是首个在該视觉识别挑战赛中超越人类表现的已发表案例。[22] 中的分析表明，人类错误主要源自细微特征识别问题与类别认知缺失。而 [22] 中的研究则指出

这些结果表明，算法在细粒度识别任务上能够表现得更好（例如，数据集中的120种犬类）。图4的第二行展示了我们该方法成功识别的部分细粒度物体示例——“杜鹃鸟”、“匈牙利马犬”以及“黄凤仙花”。虽然人类可以轻易地将这些物体识别为鸟类、犬类和花卉，但要让大多数人分辨出它们的具体物种却并非易事。从负面情况来看，即便是对人类而言并不困难的案例，我们的算法依然会出错，尤其是那些需要理解上下文或具备高级知识的场景（例如，图5中的“聚光灯”图像）。

尽管我们的算法在这个特定的Pascal VOC数据集上取得了更优异的测试结果，但这并不能说明机器视觉在物体识别领域总体上就优于人类视觉。在识别诸如Pascal VOC数据集中的那些基础物体类别（即日常生活中的常见物体或概念）时，

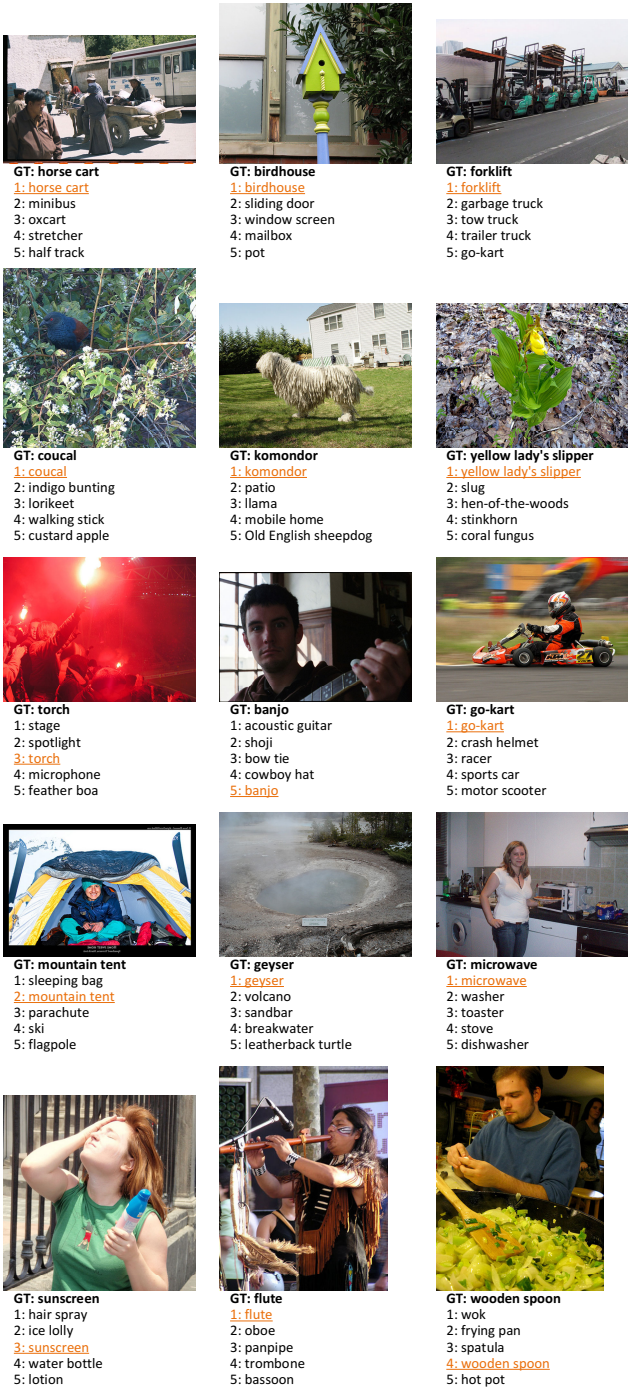


Figure 4. Example validation images successfully classified by our method. For each image, the ground-truth label and the top-5 labels predicted by our method are listed.

task [6], machines still have obvious errors in cases that are trivial for humans. Nevertheless, we believe that our results show the tremendous potential of machine algorithms to match human-level performance on visual recognition.



Figure 5. Example validation images incorrectly classified by our method, in the three classes with the highest top-5 test error. Top: “letter opener” (49% top-5 test error). Middle: “spotlight” (38%). Bottom: “restaurant” (36%). For each image, the ground-truth label and the top-5 labels predicted by our method are listed.

References

- [1] F. Agostinelli, M. Hoffman, P. Sadowski, and P. Baldi. Learning activation functions to improve deep neural networks. *arXiv:1412.6830*, 2014.
- [2] K. Chatfield, K. Simonyan, A. Vedaldi, and A. Zisserman. Return of the devil in the details: Delving deep into convolutional nets. In *BMVC*, 2014.
- [3] D. Ciresan, U. Meier, and J. Schmidhuber. Multi-column deep neural networks for image classification. In *CVPR*, 2012.
- [4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [5] D. Eigen, J. Rolfe, R. Fergus, and Y. LeCun. Understanding deep architectures using a recursive convolutional network. *arXiv:1312.1847*, 2013.
- [6] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, pages 303–338, 2010.

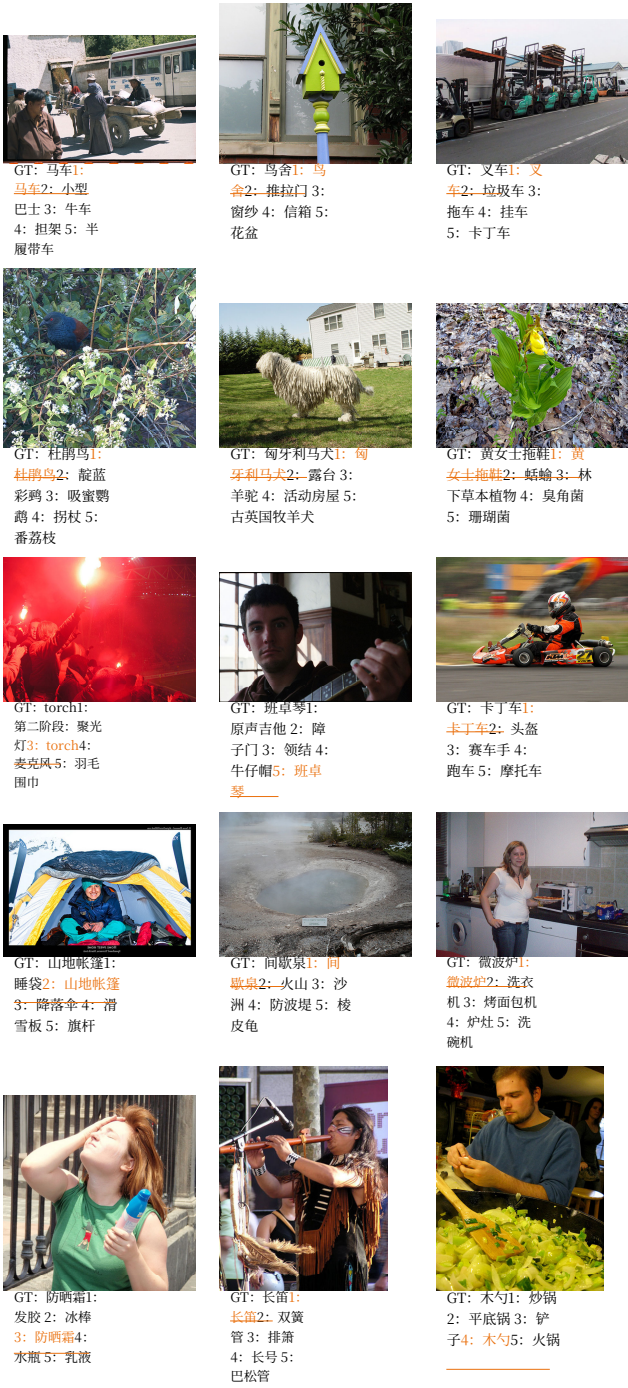


图4. 展示了一些通过我们的该方法成功分类的验证图像。对于每张图像，都会列出其真实标签以及该方法预测出的Top-5标签。

在那些对人类来说极其简单的任务中，[6]，机器仍然存在明显的错误。尽管如此，我们依然认为这些结果充分体现了机器算法在视觉识别领域达到人类水平性能的巨大潜力。



图5. 展示了那些被我们的方法错误分类的验证图像，这些图像来自Top-5测试错误率最高的三个类别。上方为“开信刀”（Top-5测试错误率为49%），中间为“聚光灯”（38%），下方为“餐厅”（36%）。每张图像同样列出了其真实标签以及该方法预测出的Top-5标签。

参考文献

- [1] F. Agostinelli, M. Hoffman, P. Sadowski以及P. Baldi共同发表了论文《通过学习激活函数提升深度神经网络性能》。 *arXiv:1412.4389*，发表时间为2014年。
- [2] K. Chatfield, K. Simonyan, A. Vedaldi和A. Zisserman则撰文《细节中的隐患：深入探究卷积网络》，该文发表于*BMVC*，时间同样为2014年。
- [3] D. Ciresan, U. Meier和J. Schmidhuber提出了用于图像分类的多列深度神经网络，相关研究发表在*CVPR*，时间为2012年。
- [4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li和L. Fei-Fei创建了大规模分层图像数据库Imagenet，该论文发表于*CVPR*，时间为2009年。
- [5] D. Eigen, J. Rolfe, R. Fergus和Y. LeCun利用递归卷积神经网络来解析深度架构，相关成果发表在*arXiv:1312.1847*，时间为2013年。
- [6] M. Everingham, L. Van Gool, C. K. Williams, J. Winn和A. Zisserman介绍了Pascal视觉物体类别挑战赛，该文章刊登在*IJCV*上，页码为303–338，出版时间为2010年。

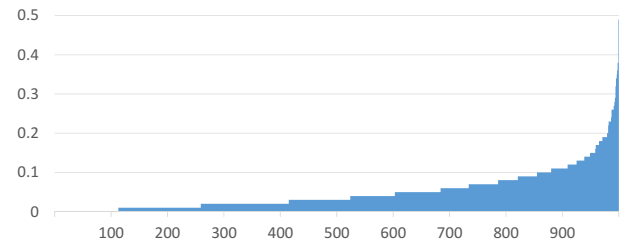


Figure 6. The per-class top-5 errors of our result (average of 4.94%) on the test set. Errors are displayed in ascending order.

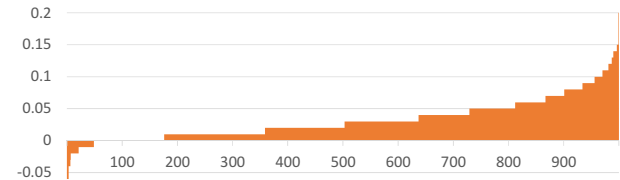


Figure 7. The difference of top-5 error rates between our result (average of 4.94%) and our team's in-competition result for ILSVRC 2014 (average of 8.06%) on the test set, displayed in ascending order. A positive number indicates a reduced error rate.

- [7] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics*, pages 249–256, 2010.
- [8] X. Glorot, A. Bordes, and Y. Bengio. Deep sparse rectifier networks. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, pages 315–323, 2011.
- [9] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv:1302.4389*, 2013.
- [10] K. He and J. Sun. Convolutional neural networks at constrained time cost. *arXiv:1412.1710*, 2014.
- [11] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *arXiv:1406.4729v2*, 2014.
- [12] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv:1207.0580*, 2012.
- [13] A. G. Howard. Some improvements on deep convolutional neural network based image classification. *arXiv:1312.5402*, 2013.
- [14] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv:1408.5093*, 2014.
- [15] A. Krizhevsky. One weird trick for parallelizing convolutional neural networks. *arXiv:1404.5997*, 2014.
- [16] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [17] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation

applied to handwritten zip code recognition. *Neural computation*, 1989.

- [18] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-supervised nets. *arXiv:1409.5185*, 2014.
- [19] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv:1312.4400*, 2013.
- [20] A. L. Maas, A. Y. Hannun, and A. Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *ICML*, 2013.
- [21] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, pages 807–814, 2010.
- [22] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *arXiv:1409.0575*, 2014.
- [23] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv:1312.6120*, 2013.
- [24] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. 2014.
- [25] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556*, 2014.
- [26] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, pages 1929–1958, 2014.
- [27] R. K. Srivastava, J. Masci, S. Kazerounian, F. Gomez, and J. Schmidhuber. Compete to compute. In *NIPS*, pages 2310–2318, 2013.
- [28] Y. Sun, Y. Chen, X. Wang, and X. Tang. Deep learning face representation by joint identification-verification. In *NIPS*, 2014.
- [29] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. *arXiv:1409.4842*, 2014.
- [30] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf. Deepface: Closing the gap to human-level performance in face verification. In *CVPR*, 2014.
- [31] L. Wan, M. Zeiler, S. Zhang, Y. L. Cun, and R. Fergus. Regularization of neural networks using dropconnect. In *ICML*, pages 1058–1066, 2013.
- [32] R. Wu, S. Yan, Y. Shan, Q. Dang, and G. Sun. Deep image: Scaling up image recognition. *arXiv:1501.02876*, 2015.
- [33] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. In *ECCV*, 2014.
- [34] M. D. Zeiler, M. Ranzato, R. Monga, M. Mao, K. Yang, Q. V. Le, P. Nguyen, A. Senior, V. Vanhoucke, J. Dean, and G. E. Hinton. On rectified linear units for speech processing. In *ICASSP*, 2013.

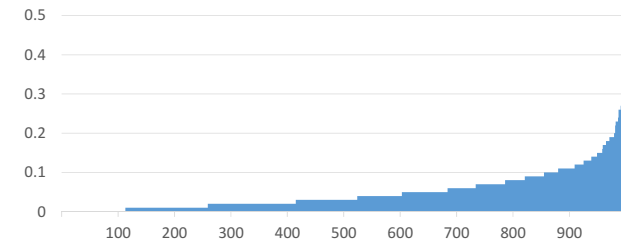


图6. 测试集上我们方法的每类前5错误率（平均值为4.94%），误差值按从小到大的顺序排列。

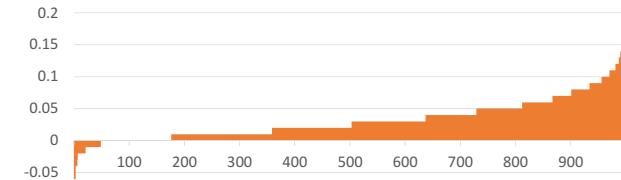


图7. 测试集上我们方法的Top-5错误率（平均值4.94%）与我们的团队在2014年ILSVRC竞赛中所取得的结果（平均值8.06%）之间的差异，同样以升序展示；正数代表错误率更低。

[7] X. Glorot与Y. Bengio在《国际人工智能与统计会议》上发表了论文《理解训练深度前馈神经网络的难度》，该文页码为249–256，发表时间为2010年。[8] X. Glorot、A. Bordes和Y. Bengio发表了关于深度稀疏整流器网络的论文，刊登在《第14届国际人工智能与统计会议论文集》中，页码为315–323，出版时间为2011年。[9] I. J. Goodfellow、D. Warde-Farley、M. Mirza、A. Courville和Y. Bengio提出了Maxout网络，相关研究发表在*arXiv:1302.4389*，时间为2013年。[10] K. He和J. Sun研究了时间约束下的卷积神经网络，论文发表在*arXiv:1412.1710*，时间为2014年。[11] K. He、X. Zhang、S. Ren和J. Sun探讨了深度卷积网络中用于视觉识别的空间金字塔池化技术，相关成果发表在*arXiv:1406.4729v2*，时间为2014年。[12] G. E. Hinton、N. Srivastava、A. Krizhevsky、I. Sutskever和R. R. Salakhutdinov提出了通过防止特征检测器协同适应来优化神经网络的方法，该论文发表在*arXiv:12070580*，时间为2012年。[13] A. G. Howard对基于深度卷积神经网络的图像分类方法进行了一些改进，相关研究发表在*arXiv:13125402*，时间为2013年。[14] Y. Jia、E. Shelhamer、J. Donahue、S. Karayev、J. Long、R. Girshick、S. Guadarrama和T. Darrell开发了Caffe框架，这是一种用于快速特征嵌入的卷积架构，相关论文发表在*arXiv:14085093*，时间为2014年。[15] A. Krizhevsky分享了一种并行化卷积神经网络的有效技巧，该论文发表在*arXiv:1404.5997*，时间为2014年。[16] A. Krizhevsky、I. Sutskever和G. Hinton研究了使用深度卷积神经网络进行ImageNet分类的方法，相关成果发表在*NIPS*，时间为2012年。[17] Y. LeCun、B. Boser、J. S. Denker、D. Henderson、R. E. Howard、W. Hubbard和L. D. Jackel撰写了关于反向传播的文章。

该技术还被应用于手写邮政编码识别领域。*Neural computation*，1989年。[18] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang和Z. Tu所著的《深度监督网络》。*arXiv:1409.5185*，2014年。[19] M. Lin, Q. Chen和S. Yan所著的《网络中的网络》。*arXiv:1312.4400*，2013年。[20] A. L. Maas, A. Y. Hannun和A. Y. Ng所著的《整流非线性特性可提升神经网络声学模型性能》。载于*ICML*，2013年。[21] V. Nair和G. E. Hinton所著的《整流线性单元能优化受限玻尔兹曼机性能》。载于*ICML*，页码807–814，2010年。[22] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein等人所著的《Imagenet大规模视觉识别挑战赛》。*arXiv:1409.0575*，2014年。[23] A. M. Saxe, J. L. McClelland和S. Ganguli所著的《深度线性神经网络中学习非线性动力学的精确解》。*arXiv:1312.6120*，2013年。[24] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus和Y. LeCun所著的《Overfeat：基于卷积网络的集成式识别、定位与检测方法》，2014年。[25] K. Simonyan和A. Zisserman所著的《用于大规模图像识别的超深层卷积网络》。*arXiv:1409.1556*，2014年。[26] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever和R. Salakhutdinov所著的《Dropout：一种防止神经网络过拟合的简单方法》。载于机器学习研究期刊，页码1929–1958，2014年。[27] R. K. Srivastava, J. Masci, S. Kazerounian, F. Gomez和J. Schmidhuber所著的《竞争计算算法》。载于*NIPS*，页码2310–2318，2013年。[28] Y. Sun, Y. Chen, X. Wang和X. Tang所著的《通过联合识别与验证实现深度学习人脸表示方法》。载于*NIPS*，2014年。[29] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke和A. Rabinovich所著的《借助卷积操作进一步深化模型结构》。*arXiv:1409.4842*，2014年。[30] Y. Taigman, M. Yang, M. Ranzato和L. Wolf所著的《Deepface：缩小人脸识别领域与人类水平性能之间的差距》。载于*CVPR*，2014年。[31] L. Wan, M. Zeiler, S. Zhang, Y. L. Cun和R. Fergus所著的《利用dropconnect正则化方法对神经网络进行优化》。载于*ICML*，页码1058–1066，2013年。[32] R. Wu, S. Yan, Y. Shan, Q. Dang和G. Sun所著的《Deep Image：提升图像识别任务的性能水平》。*arXiv:1501.02876*，2015年。[33] M. D. Zeiler和R. Fergus所著的《可视化与理解卷积神经网络》。载于欧洲计算机视觉大会，2014年。[34] M. D. Zeiler, M. Ranzato, R. Monga, M. Mao, K. Yang, Q. V. Le, P. Nguyen, A. Senior, V. Vanhoucke, J. Dean和G. E. Hinton所著的《用于语音处理的整流线性单元研究》。载于*ICASSP*，2013年。